Sheffield Hallam University

College of Business, Technology and Engineering

MSc Computing Research Project Report

Real-Time Monitoring Anomaly Detection in Bitcoin Transactions
Using Big Data Tools, Machine Learning and Dashboard

A dissertation submitted in partial fulfilment of the requirements of Sheffield Hallam University for the degree of Master of Science in Big Data Analytics

Student Name	JOSEPH ARAGBAIYE
Student ID	C5025912
Supervisor	JAMES BALDWIN
Date of Submission	10 SEPTEMBER 2026

STATEMENT: This dissertation does NOT contain confidential material and may be made available to staff and students via the library.

Acknowledgements

I would like to thank the staff of Sheffield Hallam University and the lecturers who supported my Big Data Analytics studies. I am especially grateful to my supervisor, James Baldwin, for his guidance, encouragement and advice throughout this project. I also thank my friend at the National Institute of Information Technology, Joseph Musa, for voluntarily reviewing and evaluating the project outcome. Their support and pushes helped me complete this research project.

Abstract

Bitcoin transactions change constantly, but reliable fraud labels are limited. This project therefore treats suspicious activity as an anomaly-detection problem rather than assuming every unusual transaction is fraud. The system captures Bitcoin transaction data through Bitcoin Core and ZeroMQ/RPC, passes records through Apache Kafka, prepares five transaction features and scores each transaction with three unsupervised models: Half-Space Trees, Online Isolation Forest and Batch Isolation Forest. ADWIN watches the feature stream for change, Mini-Batch K-Means adds behavioural context, and a two-out-of-three voting rule requires majority model support. SQLite stores the results and Grafana displays them for investigation.

Implementation evidence showed that transactions moved through the pipeline, all three models produced scores and flags, agreement was recorded, drift states changed and stored results appeared in Grafana. HST had the lowest captured scoring latency, while the Isolation Forest models took longer and occasionally showed spikes. Because comprehensive ground-truth fraud labels were unavailable, the project does not claim verified fraud-classification accuracy. Its main contribution is the practical integration and evaluation of streaming ingestion, unsupervised detectors, drift handling, behavioural context, explainability, persistence and visual monitoring in one auditable Bitcoin analytics pipeline.

Main-text word count: 3,000 words (Chapters 1-6, excluding headings, tables, figure captions, references and appendices).

Table of Contents

CHAPTER 1 - INTRODUCTION
1.1 Background and Context
1.2 Research Problem
1.3 Research Question
1.4 Research Aim
1.5 Research Objectives
1.6 Rationale
1.7 Project Deliverable
CHAPTER 2 - LITERATURE REVIEW
2.1 Literature Review Strategy
2.2 Search Queries and Search Results
2.3 Bitcoin Transaction Fraud and Anomaly Detection
2.4 Offline versus Real-Time Bitcoin Analysis
2.5 Streaming Big-Data Architectures
2.6 Unsupervised Anomaly Detection
2.7 Isolation Forest
2.8 Half-Space Trees
2.9 Online Isolation Forest
2.10 Concept Drift and ADWIN
2.11 Ensemble and Consensus Anomaly Detection
2.12 Explainability and Behavioural Context
2.13 Literature Gaps
2.14 Course of Action Derived from the Literature
CHAPTER 3 - PROJECT DESIGN
3.1 Project Requirements
3.2 High-Level Architecture
3.3 Low-Level Design
3.4 Feature Engineering Design
3.5 Hybrid ML and Decision Logic
3.6 Data Storage and Dashboard Design
3.7 System Test Cases
3.8 Risk Assessment
3.9 Dataset / Data Source
3.10 Ethics and Ethics Approval
3.11 Repository and Reproducibility
CHAPTER 4 - RESULTS
4.1 Evidence of End-to-End Implementation
4.2 Model Outputs
4.3 Latency Results
4.4 Concept Drift Results
4.5 Behavioral / Wallet Results
4.6 Grafana Results
4.7 Alignment with the Research Question
CHAPTER 5 - EVALUATION
5.1 Technical Evaluation
5.2 User Acceptance Testing / Beneficiary Validation
5.3 Participant Demographics and Selection Rationale
5.4 Consent and Participation Evidence
5.5 Feedback
5.6 Changes Made after Evaluation
5.7 Critical Self-Evaluation
5.8 Evaluation against Objectives
CHAPTER 6 - CONCLUSION
6.1 Summary of Project Achievements
6.2 Achievement of Objectives
6.3 Contribution and Practical Value
6.4 Limitations
6.5 Future Prospects
REFERENCES
APPENDICES

List of Figures

Figure 1. Bitcoin Core local-node status returned by bitcoin-cli getblockchaininfo.
Figure 2. Structured Bitcoin transaction records observed in the live processing stream.
Figure 3. SQLite transaction-verdict records showing verdict, risk score and anomaly votes.
Figure 4. Grafana server service shown as active (running), with the SQLite datasource plugin loaded.
Figure 5. Grafana live decision view showing a CRITICAL verdict with two of three models flagged.
Figure 6. Grafana model-flag panel querying the latest HST, Online IF and Batch IF flags from SQLite.
Figure 7. Grafana consensus-vote panel showing anomaly votes, normal votes and available models.
Figure 8. Model anomaly-flag counts for HST, Online Isolation Forest and Batch Isolation Forest.
Figure 9. Verdict distribution for the latest 1,000 analysed transactions.
Figure 10. Per-model scoring latency over time.
Figure 10a. Logarithmic comparison of captured and maximum per-model scoring latency.
Figure 11. Histogram comparing observed latency distributions.
Figure 12. Adaptive-controller states during stable, calibrating, strict, sensitive and drift-adaptation operation.
Figure 13. Grafana adaptation-mode panel showing DRIFT ADAPTATION.
Figure 14. Actionable alert and drift feed showing verdicts, consensus, flagged models and drift state.
Figure 15. Investigator transaction ledger exposing latency, engineered features, behavioural profile and explanations.
Figure 16. Risk score versus Bitcoin amount scatter plot.
Figure 17. Investigator ledger showing model scores, percentiles, thresholds, flags and health fields.
Figure 18. Grafana monitoring dashboard summarising risk, model latency, anomaly flags, consensus and adaptive-controller activity.

List of Tables

Table 1. Project requirements traceability.
Table 2. System test cases and evidence status.
Table 3. Project risk register.
Table 4. Evaluation against project objectives.

List of Abbreviations

Abbreviation	Meaning
BTC	Bitcoin
ZMQ	ZeroMQ
RPC	Remote Procedure Call
ML	Machine Learning
HST	Half-Space Trees
IF	Isolation Forest
ADWIN	Adaptive Windowing
SHAP	SHapley Additive exPlanations
UAT	User Acceptance Testing
API	Application Programming Interface

CHAPTER 1 - INTRODUCTION

1.1 Background and Context

Bitcoin was introduced by Satoshi Nakamoto as a peer-to-peer electronic cash system that allows value transfer without a bank or central organisation (Nakamoto, 2008). Transactions are recorded on the public blockchain. Before confirmation, a transaction normally waits in the mempool, giving analysts an early opportunity to inspect unusual activity.

1.2 Research Problem

A large part of the research on Bitcoin anomaly detection has been carried out using historical transactions that were already confirmed **(Weber et al., 2019; Cholevas et al., 2024).** This makes the experiments easier to reproduce, although it says less about how a system would perform while transactions are still waiting for confirmation. Bitcoin transaction patterns are also not fixed and may change over time, so findings based on older data may gradually become less representative **(Gama et al., 2014).** Recent reviews continue to point to real-time and adaptive blockchain anomaly detection as an area that needs further work **(Shevchuk et al., 2025).** For this reason, the project monitors transaction amount, virtual size, input and output structure, as well as changes in these features as new transactions appear. Any alert is treated as a reason for further checking rather than as proof of fraud.

1.3 Research Question

How can a real-time big-data streaming architecture and adaptive unsupervised machine-learning methods support scalable anomaly detection in high-velocity Bitcoin transaction streams?

1.4 Research Aim

This project aims to design, build and evaluate a real-time Bitcoin mempool anomaly-detection pipeline using big-data streaming and adaptive unsupervised machine-learning methods.

1.5 Research Objectives

- Critically review and justify suitable Bitcoin anomaly-detection methods, tools and techniques.
- Build a Bitcoin Core-ZeroMQ/RPC-Kafka architecture for continuous transaction ingestion.]
- Engineer features and compare HST, Online Isolation Forest and Batch Isolation Forest.
- Implement and justify drift detection, model consensus, behavioral clustering and explainability.
- Evaluate operation, latency, agreement, adaptation, storage and visual monitoring.
- Critically assess limitations, risks and reproducibility without treating anomalies as confirmed fraud.

1.6 Rationale

The rationale is to address limitations of static historical analysis in a changing, weakly labelled environment. Mempool monitoring provides earlier visibility before confirmation, while unsupervised learning can identify unusual patterns without a known fraud class. Online models and ADWIN address changing behaviour; model consensus reduces dependence on one detector; clustering and SHAP add context. The contribution is therefore an end-to-end, inspectable monitoring pipeline and an evaluation of how established components operate together, rather than a new machine-learning algorithm. The integrated design also makes trade-offs visible: HST can be compared with slower isolation models, drift can be distinguished from isolated disagreement, and stored evidence can be reviewed after processing. This supports technical evaluation and cybersecurity monitoring while keeping anomaly evidence separate from a fraud judgement.

1.7 Project Deliverable

The deliverable connects Bitcoin Core, ZeroMQ/RPC, Kafka, feature engineering, three anomaly detectors, ADWIN, Mini-Batch K-Means, ensemble logic, SQLite and Grafana. It records scores, risk,

disagreement, drift, behaviour, explanations and latency so decisions can be inspected rather than treated as unexplained fraud alerts.

CHAPTER 2 - LITERATURE REVIEW

2.1 Literature Review Strategy

The review used Google Scholar, CORE, Semantic Scholar, arXiv, DOAJ, ResearchGate and Sheffield Hallam university-library access. Searches covered Bitcoin, blockchain, anomaly detection, streaming, unsupervised learning, drift, ensembles and explainable AI. Sources were retained when they justified a system component, research decision or identified limitation; original publications were used for the final discussion.

2.2 Search Queries and Search Results

Example searches included “Bitcoin transaction anomaly detection”, “Bitcoin mempool anomaly detection”, “Half-Space Trees streaming anomaly detection”, “Online Isolation Forest cryptocurrency”, “ADWIN transaction stream”, “ensemble anomaly detection blockchain”, “Kafka Bitcoin streaming” and “SHAP Bitcoin fraud detection”. Results were screened for relevance to live processing, modelling, drift, explainability, storage or monitoring; duplicates and unrelated items were removed.

2.3 Bitcoin Transaction Fraud and Anomaly Detection

Bitcoin is pseudonymous: transactions and addresses are public, but ownership is usually unknown, making reliable fraud labels difficult. The Elliptic dataset illustrates the coexistence of labelled and unlabelled Bitcoin transactions (Weber et al., 2019). Anomaly detection identifies deviations from expected behaviour; it does not prove fraud (Chandola, Banerjee, & Kumar, 2009). Pham and Lee (2016) used unsupervised methods to identify unusual Bitcoin activity, supporting this approach when labels are limited.

2.4 Offline versus Real-Time Bitcoin Analysis

Much cryptocurrency research uses historical confirmed transactions. Weber et al. (2019), for example, analysed more than 200,000 Bitcoin transactions across time steps. Historical data is repeatable but cannot demonstrate pre-confirmation operation. This project does not claim real-time analysis as new; it examines whether live ingestion, online and periodically refreshed detectors, drift handling, context, explanation and monitoring can operate together which also defines an innovative initiative that marvels in technical novelty.

2.5 Streaming Big-Data Architectures

Real-time anomaly detection requires continuous processing of an unbounded stream, where latency and correctness matter (Akidau et al., 2015). Kafka separates producers and consumers and supports partitioned high-volume messaging (Kreps, Narkhede, & Rao, 2011; Raptis & Passarella, 2022). Here, Bitcoin Core ZeroMQ provides notifications and RPC supplies transaction details. Because ZeroMQ notifications can be missed (Bitcoin Core, 2026), Kafka adds buffering and decouples collection from model processing.

2.6 Unsupervised Anomaly Detection

Unsupervised anomaly detection is useful when dependable fraud labels are unavailable. Isolation methods partition rare observations; HST monitors low-mass regions; distance and density methods compare separation or neighbourhood density (Breunig et al., 2000). Tree-based methods were preferred for continuous processing because they can score efficiently, while using more than one detector reduces dependence on a single modelling assumption.

2.7 Isolation Forest

Isolation Forest isolates observations through random feature splits. Unusual points tend to require shorter paths than dense normal observations (Liu, Ting, & Zhou, 2008). It works without fraud labels, but a batch model stays fixed after training. In a changing stream it can therefore become stale until retrained, which motivates comparison with online approaches.

2.8 Half-Space Trees

Half-Space Trees were designed for streaming anomaly detection and track how much data enters random regions of feature space (Tan, Ting, & Liu, 2011). A low-mass region is unusual because few recent observations occupy it, consistent with general anomaly-detection reasoning (Chandola et al., 2009). For example, a transaction combining unusually high value, size and input/output counts may enter a rarely occupied region. HST is used as one fast detector, not a final authority.

2.9 Online Isolation Forest

Online Isolation Forest adapts as observations arrive, unlike Batch Isolation Forest, which remains fixed until retraining. Ding and Fei (2013) described a sliding-window Isolation Forest in which recent data replaces older information. Online adaptation may remain closer to current behaviour but adds processing work. Comparing online and batch isolation under the same stream therefore exposes the trade-off between responsiveness and latency.

2.10 Concept Drift and ADWIN

Concept drift occurs when stream behaviour changes, making older assumptions less useful (Gama et al., 2014). DDM monitors classifier error rate (Gama et al., 2004), while EDDM monitors distances between classification errors (Baena-García et al., 2006); both fit supervised settings more naturally because they depend on errors. ADWIN was selected because it can monitor numeric streams without immediate fraud labels and automatically adjusts its effective window when change is detected (Bifet & Gavaldà, 2007). Separate ADWIN detectors monitor the engineered features and move the controller into drift adaptation. ADWIN is not claimed to be universally superior, only a better fit for this unlabelled streaming design.

2.11 Ensemble and Consensus Anomaly Detection

Different anomaly detectors can disagree because they model outlierness differently (Zimek, Campello, & Sander, 2014). This project combines HST, Online Isolation Forest and Batch Isolation Forest using V=I_HST+I_OIF+I_BIF, with majority support when V>=2. Zero votes means normal agreement, one means disagreement, two means majority support and three means complete agreement. Stojanović and Božić (2022) provide a close precedent: three different financial anomaly detectors generated an alert when at least two agreed. The exact HST-Online IF-Batch IF combination remains this project's design choice. Majority agreement strengthens anomaly evidence but does not prove fraud.

2.12 Explainability and Behavioural Context

An anomaly score has limited value without context. SHAP provides feature-level contributions showing which variables influenced a model output (Lundberg & Lee, 2017); here it identifies the strongest Batch Isolation Forest contribution. SHAP is not claimed to be universally best. DIFFI is an Isolation-Forest-specific alternative providing global and local importance (Carletti, Terzi, & Susto, 2023). SHAP was retained because its feature-contribution format integrates clearly with the investigator view. Mini-Batch K-Means adds separate behavioural grouping through incremental updates (Sculley, 2010); clusters provide context but do not vote.

2.13 Literature Gaps

Research already covers Bitcoin analytics, anomaly detection, streaming, concept drift, ensembles and explainable machine learning (Pham & Lee, 2016; Bifet & Gavaldà, 2007; Gama et al., 2014; Zimek et al., 2014; Lundberg & Lee, 2017), but these areas are often studied separately. Historical datasets remain common (Weber et al., 2019; Cholevas et al., 2024), while fewer studies demonstrate pre-confirmation monitoring under changing conditions (Cholevas et al., 2024; Shevchuk et al., 2025). The review found limited evidence of one system combining live ingestion, online and periodically retrained unsupervised detectors, drift control, behavioural clustering, explainability, persistence and operational visualisation. HST, Isolation Forest, ADWIN, K-Means, Kafka and SHAP already exist; the contribution is their integration and operational evaluation.

2.14 Course of Action Derived from the Literature

The literature shaped the final architecture: Bitcoin Core, ZeroMQ/RPC and Kafka move transactions; HST, Online Isolation Forest and Batch Isolation Forest provide different anomaly views; ADWIN monitors change; two-out-of-three voting limits dependence on one detector; Mini-Batch K-Means adds behavioural context; SHAP explains Batch Isolation Forest; and SQLite plus Grafana preserve and display evidence. These choices directly address static analysis, weak adaptation, single-model dependence and limited operational explanation.

CHAPTER 3 - PROJECT DESIGN

3.1 Project Requirements

Requirements cover ingestion, feature preparation, anomaly scoring, drift, behavioural grouping, consensus, storage and visualisation. Non-functional requirements cover latency, repeatability, fault handling, explainability and integrity. FR1 supports continuous data movement; FR2 and FR3 create model and adaptation evidence; FR4 preserves results for investigation. Low latency is necessary for pre-confirmation monitoring, while explainability, auditability and recoverability are necessary because an anomaly requires human verification rather than automatic fraud classification. Table 1 links each requirement to supporting evidence.

ID	Type	Requirement	Evidence
FR1	Functional	Ingest and stream Bitcoin transactions	Sections 4.1, 4.6
FR2	Functional	Engineer features and score three anomaly models	Sections 4.2, 4.5
FR3	Functional	Detect drift and combine model decisions	Sections 4.2, 4.4
FR4	Functional	Persist outputs and visualise alerts	Sections 4.1, 4.6
NFR1	Non-functional	Maintain low model latency	Section 4.3
NFR2	Non-functional	Support explainability and auditability	Sections 4.5, 4.6
NFR3	Non-functional	Preserve integrity, monitorability and recoverability	Sections 4.1, 5.1

Table 1. Project requirements traceability.

3.2 High-Level Architecture

The flow is Bitcoin Core -> ZeroMQ/RPC -> Collector -> Kafka -> Feature Engineering -> Hybrid ML -> Decision/Alert -> SQLite -> Grafana. Separating capture, processing, storage and presentation allows each stage to be monitored independently.

3.3 Low-Level Design

The Kafka consumer checks duplicates, prepares features, detects drift, scores models, updates online learning and Mini-Batch K-Means, calculates consensus and risk, selects a verdict and stores the evidence.

3.4 Feature Engineering Design

The models use amount, virtual size, input count, output count and output-to-input ratio. Log transformations reduce skew and HST receives bounded features. Values remain available for explanation.

3.5 Hybrid ML and Decision Logic

HST, Online Isolation Forest and Batch Isolation Forest each produce a flag; at least two are required for majority detection. Recent score percentiles contribute to weighted risk, ADWIN changes sensitivity during drift, and Mini-Batch K-Means groups behaviour without voting. The implemented mathematics is summarised below, with supporting code evidence in Appendix G.

Component	Implemented mathematics	Plain-English meaning

Feature engineering	r=(O+1)/(I+1); X=[ln(1+A), ln(1+V), ln(1+I), ln(1+O), ln(r)]	Transforms raw values and reduces large-value skew.
HST bounded input	H_j=X_j/(1+X_j), j=1...4; H_5=1/(1+e^(-X_5))	Keeps HST inputs within bounded ranges.
Rolling percentile	P(s)=(N_<s + 0.5N_=s)/N	Shows how high the score is versus recent history.
Online learning order	score(x_t) -> decide(x_t) -> learn(x_t)	Prevents a transaction changing its own score.
Batch Isolation Forest	S_BIF(x)=-score_samples(x); I_BIF=1[predict(x)=-1 OR P_BIF>=q]	Flags by built-in or adaptive percentile rule.
Mini-Batch K-Means	C(x)=arg min_k ||x-mu_k||^2; K=4; B=32	Groups similar behaviour; does not vote.
Three-model consensus	V=I_HST+I_OIF+I_BIF; Detection=1 when V>=2	Requires two or more anomaly votes.
Weighted risk	R=clip(w_H P_H+w_O P_O+w_B P_B+B,0,1); initial weights=1/3	Combines percentiles plus agreement bonus.
Adaptive thresholds	MORE_SENSITIVE q=.90 M=.65 C=.85; STABLE q=.95 M=.72 C=.88; DRIFT q=.93 M=.70 C=.90; STRICT q=.98 M=.78 C=.94	Operating mode changes cut-offs used for verdicts.

Mathematical implementation artefact. Equations reproduce the implemented decision logic; supporting terminal evidence is in Appendix G.

3.6 Data Storage and Dashboard Design

SQLite stores identifiers, features, model outputs, risk, consensus, drift, behaviour, explanations and latency. Grafana queries those records to show decisions and trends. SQLite suits the local prototype, while Section 6.4 recognises its concurrency limits for production scale.

3.7 System Test Cases

Tests cover Bitcoin Core, ZMQ/RPC, Kafka, feature extraction, model scoring, SQLite, ADWIN and Grafana. Table 2 links expected and observed outcomes to figures, while further test and implementation evidence is referenced in Appendices J and L.

ID	Component	Action	Expected	Observed	Status	Evidence
TC01	Bitcoin Core	getblockchaininfo	RPC status returned	Responded; IBD=true, progress~0.795	PARTIAL	Fig. 1
TC02	ZMQ/RPC	Receive/query	Transaction records available	Stream observed; config screenshot absent	LIMITED	Fig. 2
TC03	Kafka	Consume btc-tx	Structured records received	Records observed in stream	PASS	Fig. 2
TC04	Hybrid ML	Score transaction	Three outputs + consensus	Scores, flags and consensus visible	PASS	Figs. 5-9, 17
TC05	SQLite	Query verdict table	Persisted rows returned	Verdict, risk and votes returned	PASS	Fig. 3
TC06	ADWIN	Observe drift	Adaptation state changes	DRIFT_ADAPTATION visible	PASS	Figs. 12-14
TC07	Grafana	Query SQLite	Panels render from datasource	Service active; panels populated	PASS	Figs. 4, 18

Table 2. System test cases and evidence status.

3.8 Risk Assessment

Risks include incomplete synchronisation, Kafka failure or lag, malformed records, SQLite locking, drift, false alerts, latency spikes and data loss. Controls include validation, buffering, monitoring and recovery. Table 3 summarises the risks and Appendix K contains the supporting risk register and map.

Risk	Likelihood	Impact	Mitigation
Incomplete node synchronisation	High	High	Verify chain status; separate bootstrap/replay evidence from fully live evidence.
Kafka unavailable / lag	Medium	High	Monitor broker/consumer health, buffer safely and restart without silent loss.
SQLite locking	Medium	Medium	Use short controlled writes and retries; consider production database at scale.
Concept drift / false alerts	High	High	Use ADWIN state, conservative adaptation and model consensus.
Latency spikes	Medium	Medium	Track per-model latency and bound expensive retraining/scoring work.
Data loss / malformed records	Low	High	Validate records, retain identifiers and log failures for replay/audit.

Table 3. Project risk register.

3.9 Dataset / Data Source

The primary source is Bitcoin Core. ZeroMQ receives notifications, RPC supplies transaction details, and Kafka transports and buffers the records. Bitcoin Core was chosen because it provides direct node-level data suitable for pre-confirmation monitoring without relying on a curated historical dataset. Public pseudonymous transaction fields also match the required features without identifying wallet owners. Stored SQLite records provide repeatable analysis evidence. The absence of verified fraud labels supports the unsupervised design but limits evaluation to anomaly behaviour, operation, latency, agreement and drift rather than fraud-classification accuracy.

3.10 Ethics and Ethics Approval

The project analyses pseudonymous public Bitcoin data and does not identify wallet owners. Human evaluation followed the University's Level 2(UREC2) ethics route, with consent, anonymisation and secure handling. Supporting ethics and participation documents are linked in Appendices B-F.

3.11 Repository and Reproducibility

Reproducibility is supported by documenting Bitcoin Core, Kafka, model settings, SQLite and Grafana. Blackboard remains the institutional submission route; the repository reference is in Appendix G. Credentials are excluded. Project planning evidence is linked in Appendices H and I.

CHAPTER 4 - RESULTS

4.1 Evidence of End-to-End Implementation

Figures 1-4 show the pipeline operating end to end. Figure 1 records Bitcoin Core responding to getblockchaininfo and shows initialblockdownload=true with progress near 0.795, proving operation but not full synchronisation. Figure 2 shows structured transaction records in the processing stream. Figure 3 shows SQLite verdict, risk and vote records, confirming persistence. Figure 4 shows Grafana running with the SQLite datasource. Together they demonstrate capture, processing, storage and visualisation; additional evidence is in Appendix L.

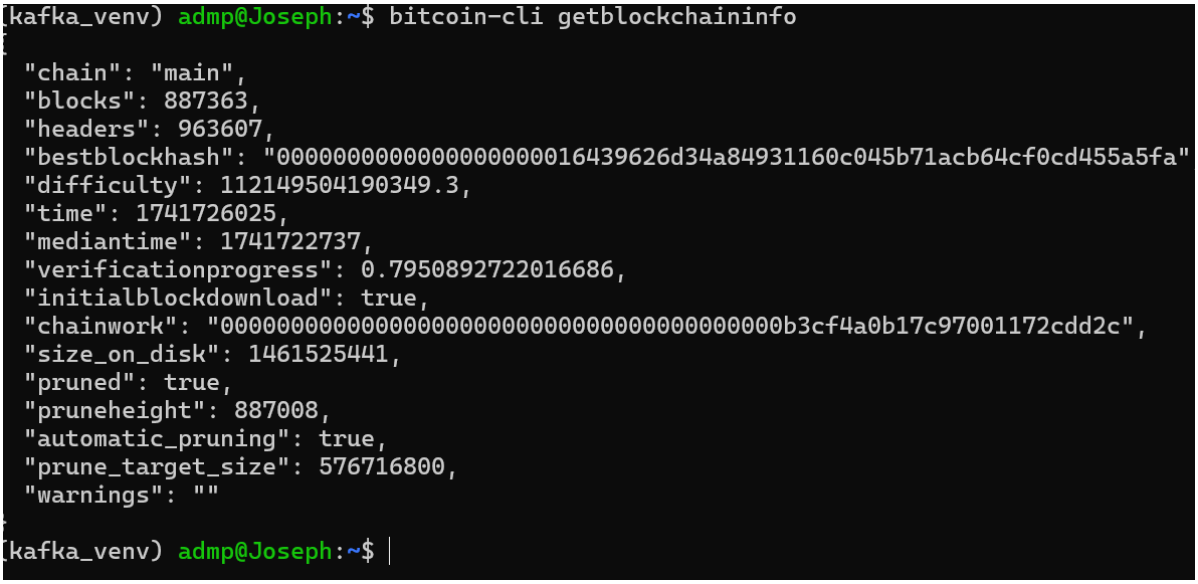


Figure 1. Bitcoin Core local-node status returned by bitcoin-cli getblockchaininfo.



Figure 2. Structured Bitcoin transaction records observed in the live processing stream.

```
admp@Joseph:~$ sqlite3 -header -column /home/admp/bitcoin-fraud-data/bitcoin_analytics.db "SELECT timestamp,substr(txid,
1,16) AS txid,verdict,risk_score,anomaly_votes FROM transaction_verdicts ORDER BY rowid DESC LIMIT 1000;"
```

timestamp	txid	verdict	risk_score	anomaly_votes
2025-03-17T23:32:58+00:00	a0a737d5128b109e	NORMAL	0.1583622	0
2025-03-17T23:32:58+00:00	97c3163d2ac80dbd	NORMAL	0.1243698	0
2025-03-17T23:32:58+00:00	c910029eb6eba5f5	NORMAL	0.1390576	0
2025-03-17T23:32:58+00:00	f4f35ad9c747b8cb	NORMAL	0.1403908	0
2025-03-17T23:32:58+00:00	fbe98a55e9ab0a8c	NORMAL	0.1656434	0
2025-03-17T22:39:38+00:00	21e4e5847f4c7142	NORMAL	0.1757164	0
2025-03-17T22:39:38+00:00	4194d004317be565	NORMAL	0.110044	0
2025-03-17T22:39:38+00:00	7246564f3ad4054f	NORMAL	0.1049318	0
2025-03-17T21:51:20+00:00	98dab8202cd23065	NORMAL	0.1204498	0
2025-03-17T21:51:20+00:00	cce52af6303baa80	NORMAL	0.1191634	0
2025-03-17T21:04:16+00:00	9b8d8b22fcfe5acf	NORMAL	0.1176812	0
2025-03-17T21:04:16+00:00	23aa132ef088e89d	NORMAL	0.123802	0
2025-03-17T21:04:16+00:00	331853624faf7609	NORMAL	0.1467368	0
2025-03-17T20:27:07+00:00	19dcb7774ac305d8	NORMAL	0.1274554	0
2025-03-17T20:27:07+00:00	ec6449343e796d30	NORMAL	0.1484132	0
2025-03-17T20:27:07+00:00	b0d476d6ed0b0aed	NORMAL	0.1324162	0
2025-03-17T20:27:07+00:00	ae302d9165324564	NORMAL	0.2716392	0
2025-03-17T19:36:41+00:00	6949cf91cce30333	NORMAL	0.2791414	0
2025-03-17T19:36:41+00:00	f179a14bc0c225ad	NORMAL	0.1471404	0
2025-03-17T19:36:41+00:00	87dc5f915465c653	NORMAL	0.2100332	0
2025-03-17T19:17:58+00:00	f56309fec56e268c	NORMAL	0.1245534	0
2025-03-17T19:17:58+00:00	088544b3b282f4b3	NORMAL	0.118967	0
2025-03-17T19:17:58+00:00	2119bd046fa39ac5	NORMAL	0.1344954	0
2025-03-17T18:51:12+00:00	64ede1419257f493	NORMAL	0.3911878	0
2025-03-17T18:51:12+00:00	2d341870420686c2	NORMAL	0.141779	0
2025-03-17T18:51:12+00:00	7a5f3e0f56ede20b	NORMAL	0.1452516	0

2025-03-14T07:10:50+00:00	39ffb0840fc9f6c7	NORMAL	0.350802533333333	0
2025-03-14T07:10:50+00:00	0a6f0e43372f156b	NORMAL	0.1414436	0
2025-03-14T07:10:50+00:00	9a67d711be7ba133	NORMAL	0.424480533333333	0
2025-03-14T07:10:50+00:00	fe2f20e33453e3ea	NORMAL	0.343646666666667	0
2025-03-14T07:10:50+00:00	b5d92f75a7cf4b03	NORMAL	0.354344533333333	0
2025-03-14T07:10:50+00:00	33792a1523b51935	NORMAL	0.401943333333333	0
2025-03-14T07:10:50+00:00	4249070064ce0cac	CRITICAL	1.0	3
2025-03-14T07:10:50+00:00	56aa07e0616534b3	CRITICAL	0.9839864	2
2025-03-14T07:10:50+00:00	69ea880aba83eafa	CRITICAL	1.0	3
2025-03-14T07:10:50+00:00	570fdca4512fefb3	CRITICAL	0.983298933333333	2
2025-03-14T07:10:50+00:00	fd162a5107c9a8cf	CRITICAL	0.978633866666667	2
2025-03-14T07:10:50+00:00	f2b9d37a97bd42a9	CRITICAL	0.977129466666667	2
2025-03-14T07:10:50+00:00	9884589da33e9a74	MODEL_DISAGREEMENT	0.933660266666667	1
2025-03-14T07:10:50+00:00	d0fce3f056022652	NORMAL	0.897260533333333	0
2025-03-14T07:10:50+00:00	33c52b94b71eb70f	CRITICAL	0.967262933333333	2
2025-03-14T07:10:50+00:00	42807bd9cbd4a0c2	CRITICAL	0.967763333333333	2
2025-03-14T07:10:50+00:00	9d2b23b8f0f081e9	CRITICAL	0.968266133333333	2
2025-03-14T07:10:50+00:00	3e5be71dc92d31f6	NORMAL	0.882690666666667	0
2025-03-14T07:10:50+00:00	762a4fa2587145e1	NORMAL	0.898679466666667	0
2025-03-14T07:10:50+00:00	9384001079a277ea	NORMAL	0.842282266666667	0
2025-03-14T07:10:50+00:00	a7acd0ee21ee41df	NORMAL	0.881821733333333	0
2025-03-14T07:10:50+00:00	3f181d3c2311b4ba	NORMAL	0.8419616	0
2025-03-14T07:10:50+00:00	3556764b31aa492e	NORMAL	0.842462	0
2025-03-14T07:10:50+00:00	85cf231dabe305f8	NORMAL	0.514613333333333	0
2025-03-14T07:10:50+00:00	f7ed1d1b92a4b8f5	NORMAL	0.528814	0
2025-03-14T07:10:50+00:00	cace82e34e6d726e	NORMAL	0.539535866666667	0
2025-03-14T07:10:50+00:00	a1844b5536bae846	NORMAL	0.540037066666667	0
2025-03-14T07:10:50+00:00	4f35bea29a3502b7	NORMAL	0.761663466666667	0
[illegible]				

2025-03-17T18:13:28+00:00	279cb2057a9b0ffb	NORMAL	0.3038532	0
2025-03-17T18:13:28+00:00	8e2111926d88e1db	MODEL_DISAGREEMENT	0.9535192	1
2025-03-17T17:18:52+00:00	b82baf393643337e	NORMAL	0.5254878	0
2025-03-17T17:18:52+00:00	287b2047f765ab55	NORMAL	0.5266608	0
2025-03-17T17:18:52+00:00	446143516c842bb1	NORMAL	0.5271634	0
2025-03-17T16:51:43+00:00	93cdc654e3cb85fe	NORMAL	0.5286716	0
2025-03-17T16:18:46+00:00	4407b3c88741861d	NORMAL	0.8254872	0
2025-03-17T16:18:46+00:00	f7a90e8b459d30a3	NORMAL	0.4777146	0
2025-03-17T14:53:20+00:00	9de251af76b9e367	NORMAL	0.5302078	0
2025-03-17T14:53:20+00:00	1f9570a33c28d124	NORMAL	0.4632436	0
2025-03-17T14:53:20+00:00	9c811988c6a01679	NORMAL	0.5607636	0
2025-03-17T14:53:20+00:00	59f1a6315c73523e	NORMAL	0.5312518	0
2025-03-17T14:53:20+00:00	15cb262a5c6ea4ac	NORMAL	0.5136304	0
2025-03-17T14:53:20+00:00	1af75bf6aeb1c52d	NORMAL	0.4884242	0
2025-03-17T14:53:20+00:00	619c2d1db1e824b4	NORMAL	0.490938	0
2025-03-17T14:53:20+00:00	ecc8af1a4f5e7b54	NORMAL	0.4781016	0
2025-03-17T14:53:20+00:00	ad67d09dbc118eed	NORMAL	0.499596	0
2025-03-17T14:53:20+00:00	8561d32e12bde70e	NORMAL	0.4892398	0
2025-03-17T14:53:20+00:00	f4f27fc3b5aab0f6	NORMAL	0.4988382	0
2025-03-17T14:53:20+00:00	2a794b0efdd4fab5	NORMAL	0.5770766	0
2025-03-17T14:53:20+00:00	cf4f9ba224e86f12	NORMAL	0.4932694	0
2025-03-17T14:53:20+00:00	3668232f54287dfc	NORMAL	0.4864146	0
2025-03-17T14:53:20+00:00	13760a635e4c3b3b	NORMAL	0.580684	0
2025-03-17T14:53:20+00:00	8cc65e28e14b066e	NORMAL	0.5858222	0
[illegible]				

Figure 3. SQLite transaction-verdict records showing verdict, risk score and anomaly votes.

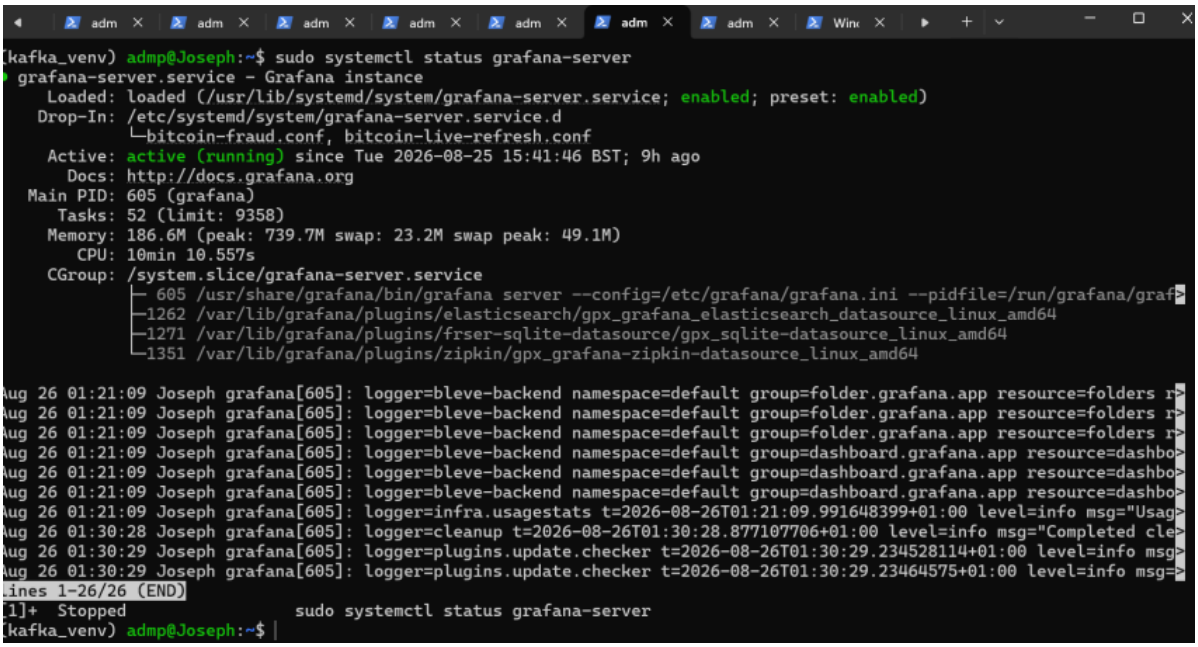


Figure 4. Grafana server service shown as active (running), with the SQLite datasource plugin loaded.

4.2 Model Outputs

Figures 5-9 and 17 show scores, flags and consensus. Figure 5 contains a CRITICAL example with two models flagged. Figures 6-7 expose individual flags and the consensus vote; Figure 8 compares model anomaly-flag counts and Figure 9 shows the latest 1,000 verdicts. Figure 17 retains scores, percentiles, thresholds and health fields. NORMAL, MODEL_DISAGREEMENT, CALIBRATING and CRITICAL records confirm that one model alone does not create a majority anomaly detection.

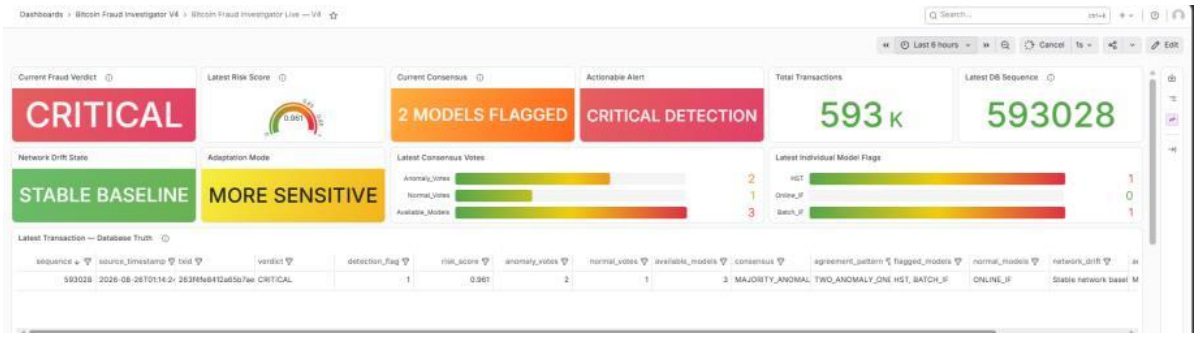


Figure 5. Grafana live decision view showing a CRITICAL verdict with two of three models flagged.

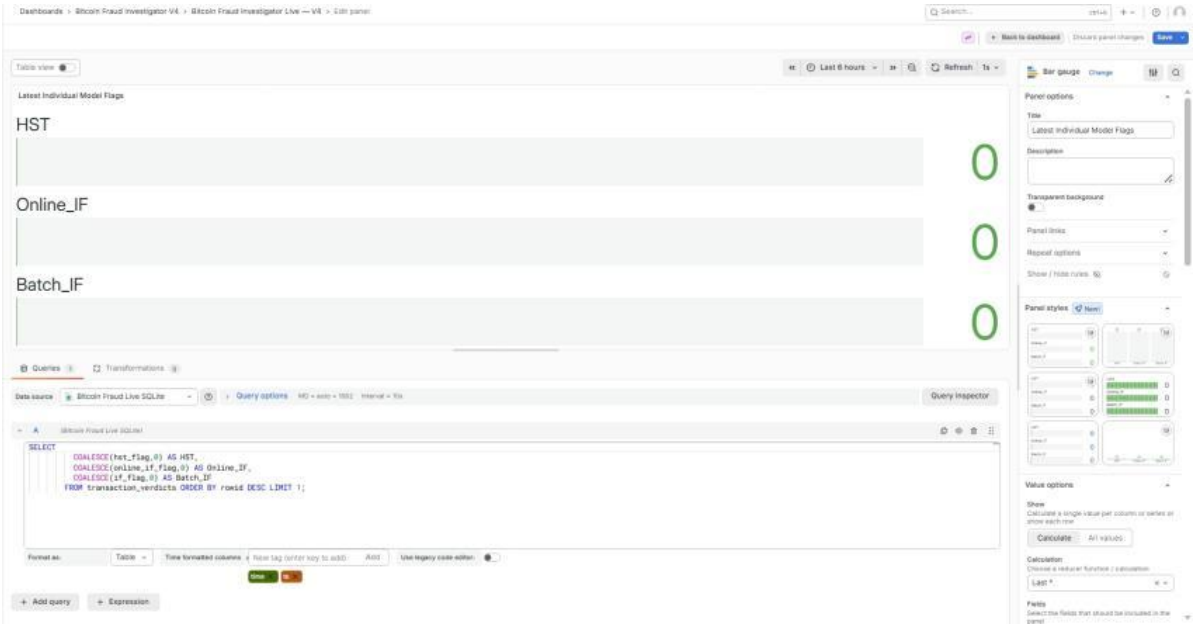


Figure 6. Grafana model-flag panel querying the latest HST, Online IF and Batch IF flags from SQLite.

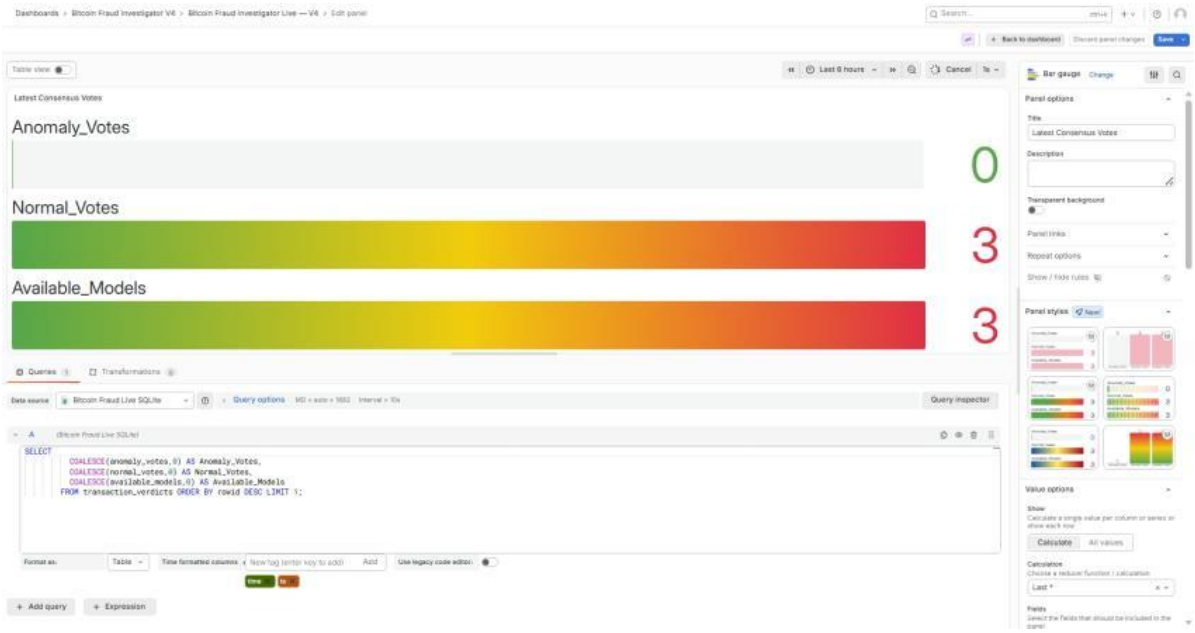


Figure 7. Grafana consensus-vote panel showing anomaly votes, normal votes and available models.



Figure 8. Model anomaly-flag counts for HST, Online Isolation Forest and Batch Isolation Forest.

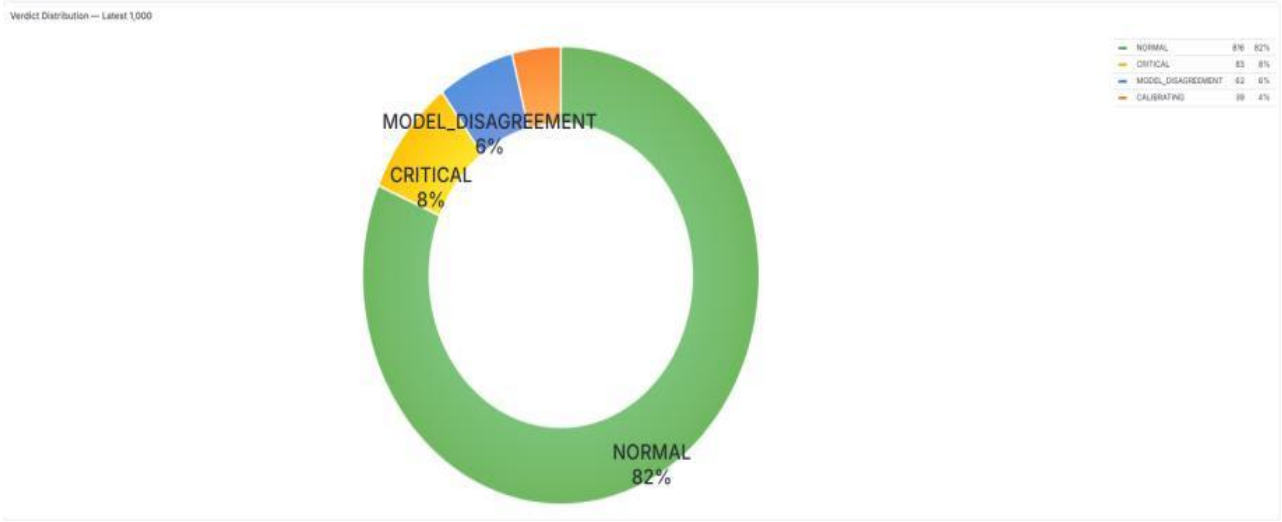


Figure 9. Verdict distribution for the latest 1,000 analysed transactions.

4.3 Latency Results

Captured latency was about 0.332 ms for HST, 11.174 ms for Online Isolation Forest and 51.899 ms for Batch Isolation Forest; maxima reached 4.554 ms, 440.606 ms and 162.241 ms. HST was fastest, while the Isolation Forest models produced occasional spikes. In Figure 10, the 440.606 ms spike stretches the linear scale and compresses smaller values. Figure 10a therefore presents the reported captured and maximum values on a logarithmic scale for clearer comparison without removing the spike. Figure 11 provides the distribution view. The maxima represent occasional delay rather than typical performance.

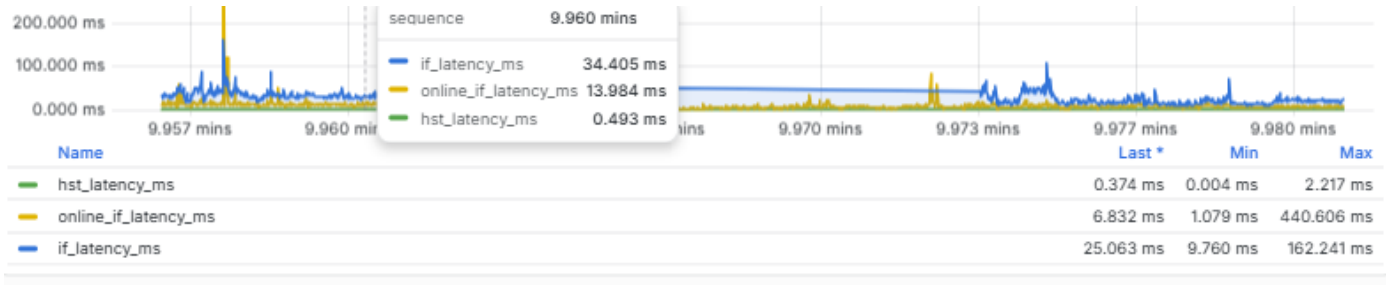


Figure 10. Per-model scoring latency over time, showing the effect of occasional spikes on the linear scale.

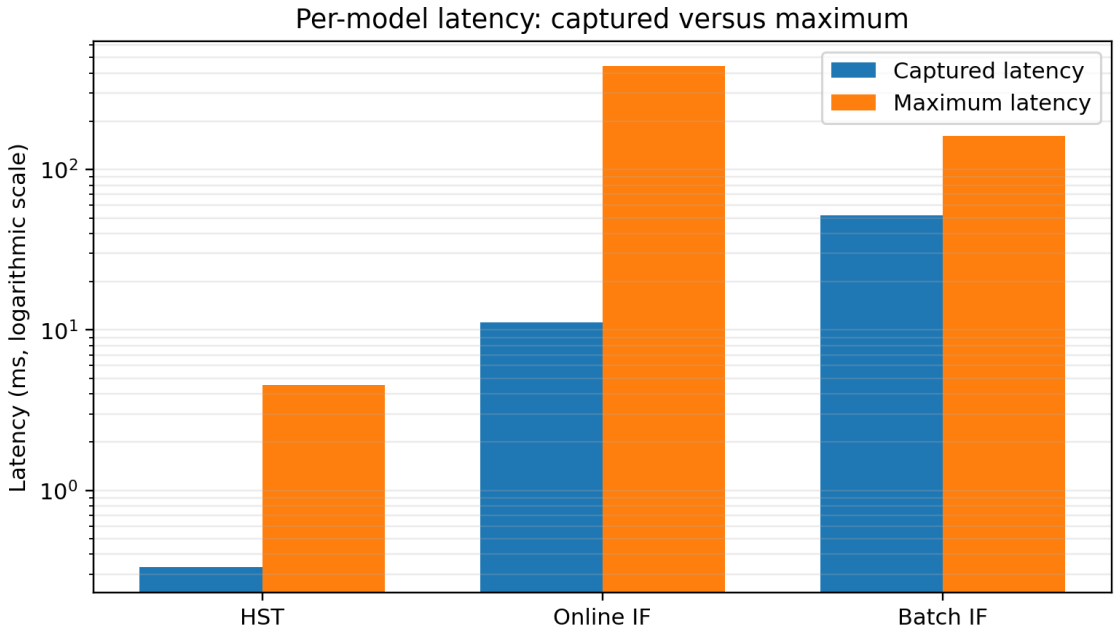


Figure 10a. Logarithmic comparison of the reported captured and maximum per-model scoring latency.

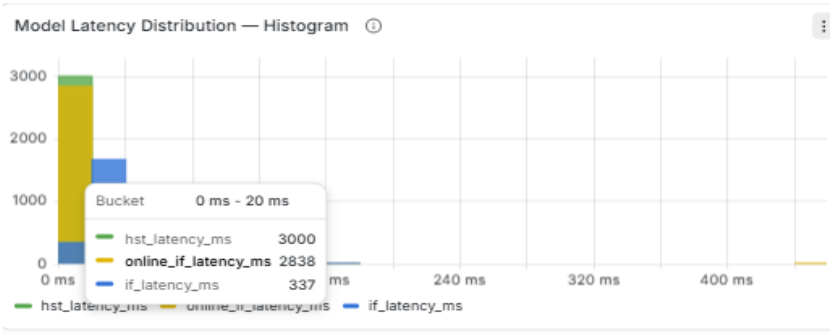


Figure 11. Histogram comparing the observed latency distributions of HST, Online Isolation Forest and Batch Isolation Forest.

4.4 Concept Drift Results

ADWIN evidence showed the controller entering DRIFT_ADAPTATION. Figure 12 summarises CALIBRATING, STABLE, MORE_SENSITIVE, STRICT and drift-adaptation states. Figure 13 shows DRIFT ADAPTATION in Grafana, while Figure 14 links drift, verdict, consensus and flagged models in the alert feed. The decision layer therefore changed sensitivity rather than using one fixed setting.

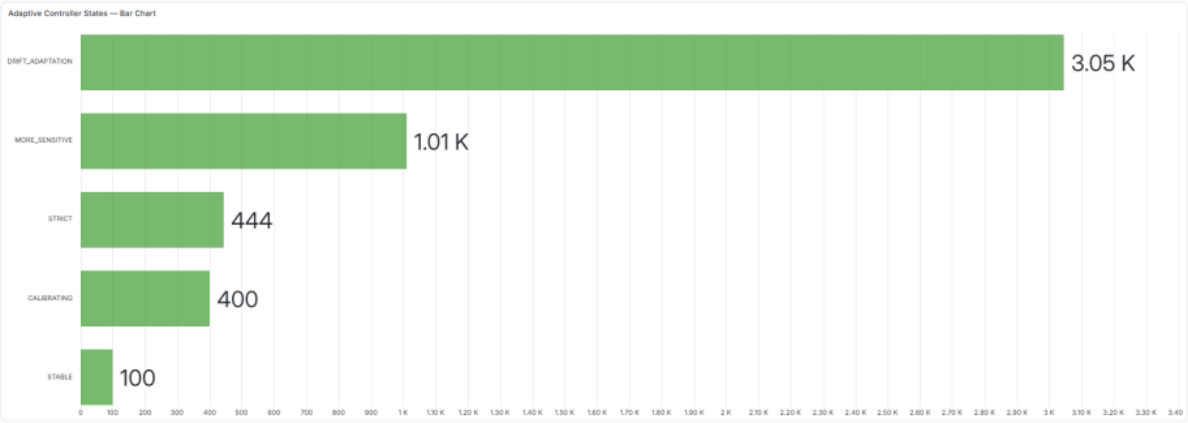


Figure 12. Adaptive-controller states recorded during stable, calibrating, strict, sensitive and drift-adaptation operation.

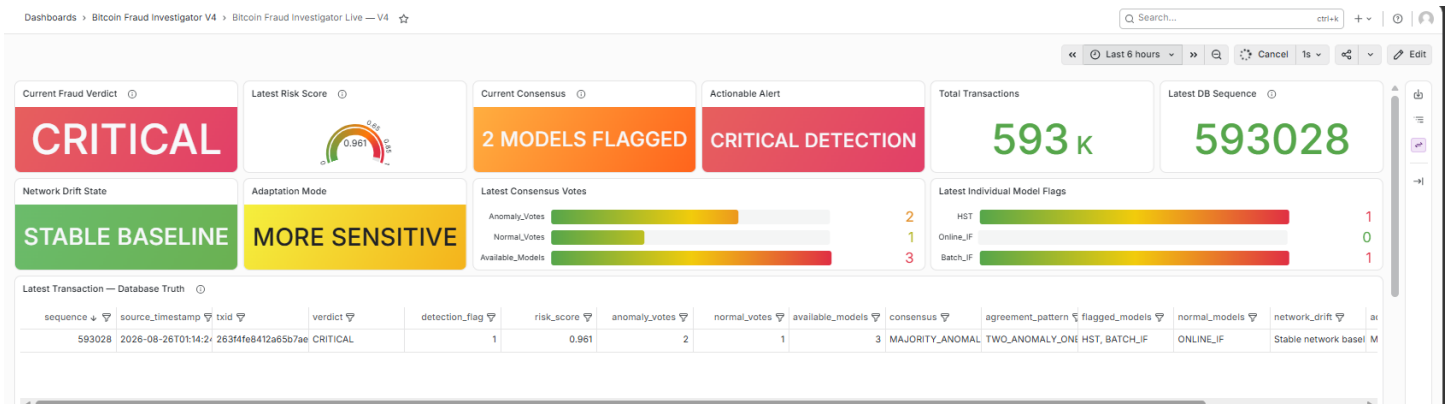


Figure 13. Grafana adaptation-mode panel showing DRIFT ADAPTATION.

network_drift	adaptation_mode	hst_ms	online_if_ms	batch_if_ms	faster_model	latency_ratio	amount_btc
Adapting after drift in	DRIFT_ADAPTATION	0.374	6.83	25.1	HST	66.9	0.00260
Adapting after drift in	DRIFT_ADAPTATION	0.367	9.06	22.2	HST	60.5	0.00225
Adapting after drift in	DRIFT_ADAPTATION	0.211	8.79	21.1	HST	100	214
Adapting after drift in	DRIFT_ADAPTATION	0.228	7.41	17.1	HST	75.3	0.00505
Adapting after drift in	DRIFT_ADAPTATION	0.179	6.57	15.0	HST	83.8	0.488
Adapting after drift in	DRIFT_ADAPTATION	0.296	6.78	21.3	HST	72.0	0.0161
Adapting after drift in	DRIFT_ADAPTATION	0.144	8.78	21.1	HST	147	0.797
Adapting after drift in	DRIFT_ADAPTATION	0.299	8.52	20.7	HST	69.1	0.00822
Adapting after drift in	DRIFT_ADAPTATION	0.295	11.2	22.8	HST	77.4	0.0309
Adapting after drift in	DRIFT_ADAPTATION	0.201	9.43	19.1	HST	95.2	1.25
Adapting after drift in	DRIFT_ADAPTATION	0.333	7.54	24.0	HST	71.9	0.00424

Figure 14. Actionable alert and drift feed showing verdicts, consensus, flagged models, drift state and adaptation mode.

4.5 Behavioural / Wallet Results

Figure 15 shows transaction features, behavioural profile, latency and explanation fields. Mini-Batch K-Means supplies behavioural context and SHAP identifies important Batch Isolation Forest features. Figure 16 shows that elevated risk is not limited to the largest Bitcoin amounts, supporting multi-feature analysis. Figure 17 exposes the model evidence needed to inspect agreement or disagreement.

network_drift	adaptation_mode	hst_ms	online_if_ms	batch_if_ms	faster_model	latency_ratio	amount_btc
Adapting after drift in	DRIFT_ADAPTATION	0.374	6.83	25.1	HST	66.9	0.00260
Adapting after drift in	DRIFT_ADAPTATION	0.367	9.06	22.2	HST	60.5	0.00225
Adapting after drift in	DRIFT_ADAPTATION	0.211	8.79	21.1	HST	100	214
Adapting after drift in	DRIFT_ADAPTATION	0.228	7.41	17.1	HST	75.3	0.00505
Adapting after drift in	DRIFT_ADAPTATION	0.179	6.57	15.0	HST	83.8	0.488
Adapting after drift in	DRIFT_ADAPTATION	0.296	6.78	21.3	HST	72.0	0.0161
Adapting after drift in	DRIFT_ADAPTATION	0.144	8.78	21.1	HST	147	0.797
Adapting after drift in	DRIFT_ADAPTATION	0.299	8.52	20.7	HST	69.1	0.00822
Adapting after drift in	DRIFT_ADAPTATION	0.295	11.2	22.8	HST	77.4	0.0309
Adapting after drift in	DRIFT_ADAPTATION	0.201	9.43	19.1	HST	95.2	1.25
Adapting after drift in	DRIFT_ADAPTATION	0.333	7.54	24.0	HST	71.9	0.00424

Figure 15. Investigator transaction ledger exposing model latency, engineered features, behavioural profile and explanation fields.

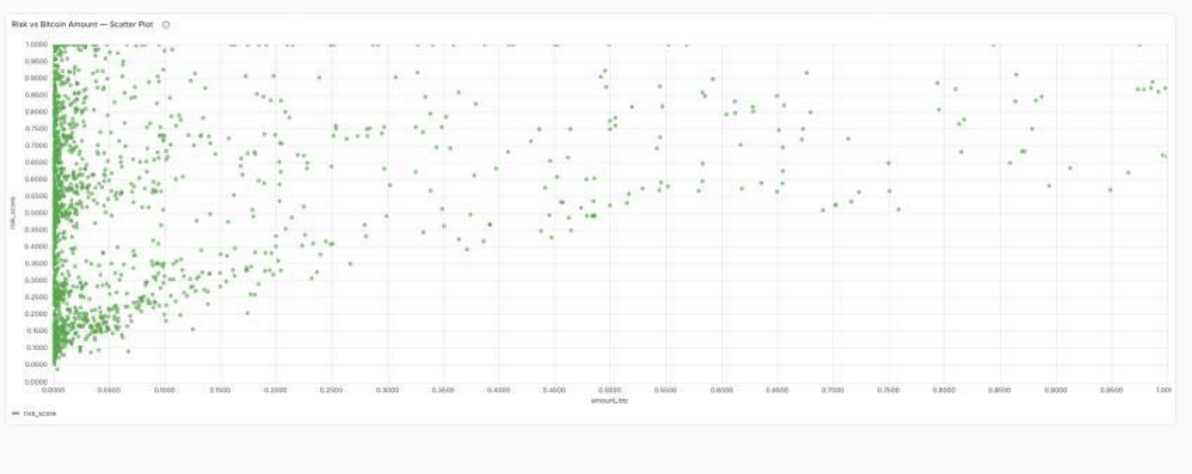


Figure 16. Risk score versus Bitcoin amount scatter plot used to inspect the relationship between value and anomaly risk.

Investigator Transaction Ledger

sequence	timestamp	txid	verdict	detection_flag	risk_score	anomaly_votes	normal_votes	available_models
598861	2026-08-26T03:53:2	58b9abc6f9f9c2aa6(	NORMAL	0	0.746	0	3	3
598860	2026-08-26T03:53:2	1d1feaebf95c17cabe8	NORMAL	0	0.330	0	3	3
598859	2026-08-26T03:53:2	fddd1be84135f85d05	CRITICAL	1	1	2	1	3
598858	2026-08-26T03:53:2	579323e35c249a81b	NORMAL	0	0.855	0	3	3
598857	2026-08-26T03:53:2	df72166005768b1282	CRITICAL	1	0.970	2	1	3
598856	2026-08-26T03:53:2	f44973d9c2f29f7001	NORMAL	0	0.772	0	3	3
598855	2026-08-26T03:53:2	d66a42b2632a1b7a0	CRITICAL	1	1	2	1	3
598854	2026-08-26T03:53:2	6f0b73f3d725603a11	NORMAL	0	0.435	0	3	3
598853	2026-08-26T03:53:2	03887d367919c2485	NORMAL	0	0.470	0	3	3
598852	2026-08-26T03:53:2	e97e36828f39c8b38	CRITICAL	1	1	2	1	3
598851	2026-08-26T03:53:2	da8336861f1c0d38d(	NORMAL	0	0.687	0	3	3

Figure 17. Investigator ledger showing HST, Online IF and Batch IF scores, percentiles, thresholds, flags and model health fields.

4.6 Grafana Results

Figure 18 combines verdicts, risk, consensus, flags, drift, latency, distributions and alerts in one dashboard. Because the panels query SQLite, the dashboard presents persisted pipeline results rather than manually prepared figures, supporting auditability and operational monitoring.

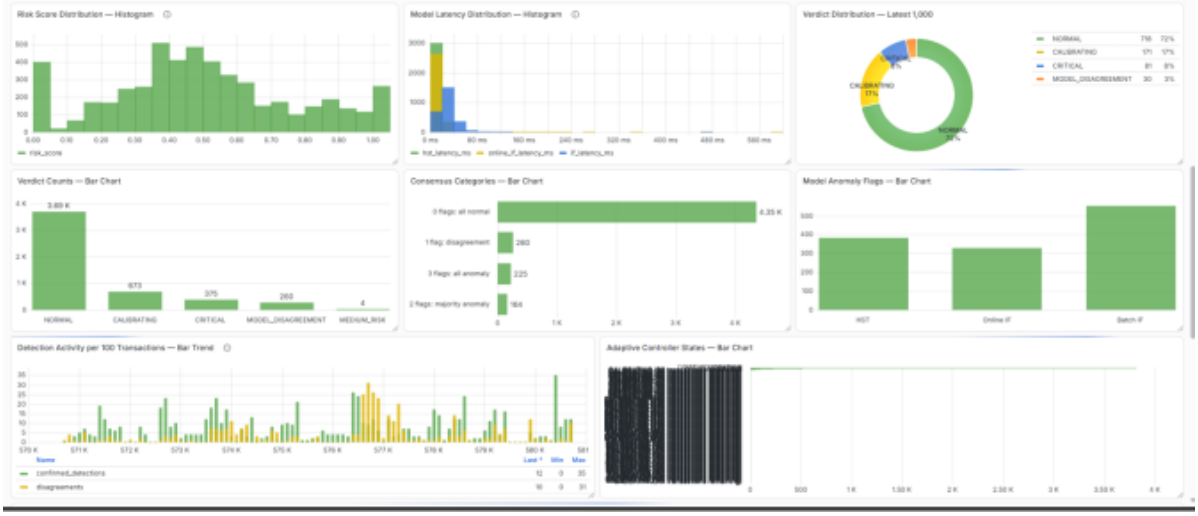


Figure 18. Grafana monitoring dashboard summarising risk, model latency, anomaly flags, consensus and adaptive-controller activity.

4.7 Alignment with the Research Question

At prototype scale, the architecture supported continuous analysis with three unsupervised detectors, drift adaptation, behavioural context, persistent evidence and monitoring. HST had very low latency, while disagreement and drift states made uncertainty visible. The design supports scalable separation of ingestion and processing, but distributed scalability was not demonstrated because deployment remained local.

CHAPTER 5 - EVALUATION

5.1 Technical Evaluation

Technical evaluation showed that transactions could be ingested, scored, checked for drift, stored and visualised. HST had the lowest captured latency; Online and Batch Isolation Forest added complementary evidence at greater processing cost (Section 4.3). MODEL_DISAGREEMENT prevented one model flag from becoming a majority detection (Section 4.2), and ADWIN altered sensitivity during change (Section 4.4). The main result is therefore operational integration, not verified fraud accuracy.

5.2 User Acceptance Testing / Beneficiary Validation

A volunteer evaluator associated with the National Institute of Information Technology reviewed whether verdicts, model agreement, drift information, explanations and investigator views were understandable and useful. Participant and evaluation materials are referenced in Appendices D-F.

5.3 Participant Demographics and Selection Rationale

Only information needed to explain the evaluator's relevance is reported. The evaluator's technical association was suitable because the system is intended for users able to interpret streaming anomaly evidence.

5.4 Consent and Participation Evidence

Human evaluation followed the Level 2 ethics process. Participant information, anonymised consent and the evaluation instrument are referenced in Appendices D-F. Bitcoin transaction data remains pseudonymous public data.

5.5 Feedback

The available report material contains no verified written feedback that can be reproduced safely. Comments, ratings or usability scores are therefore not invented; any final feedback summary must be supported by the anonymised evidence in Appendices E-F.

5.6 Changes Made after Evaluation

No change is described as evaluator-driven without evidence. Where the final evaluation record contains recommendations, the submitted version should state what was suggested, what changed and the supporting appendix evidence.

5.7 Critical Self-Evaluation

The main strength is integration: streaming, three models, drift awareness, behavioural context, explanations, persistence and monitoring operate in one flow. Limitations remain. Without ground-truth labels, anomalies are not proven fraud; SQLite may face concurrency limits; the captured node was not fully synchronised; and Isolation Forest latency spiked. These limitations restrict the claims but identify future tests.

5.8 Evaluation against Objectives

The objectives were substantially achieved: the prototype demonstrated streaming, feature engineering, three-model scoring, ADWIN adaptation, two-out-of-three consensus, behavioural profiling, SQLite storage and Grafana monitoring. Distributed performance and fraud-detection accuracy were not established because deployment remained local and verified labels were unavailable. Table 4 summarises the evidence.

Objective	Assessment	Evidence / qualification
Investigate Bitcoin anomaly-detection approaches	Achieved	Literature review and justified model selection.

Develop Bitcoin Core-ZMQ-Kafka streaming architecture	Substantially achieved	Local pipeline operated; distributed scaling not tested.
Engineer transaction and behavioural features	Achieved	Investigator ledger exposes engineered/contextual fields.
Implement and compare HST, Online IF and Batch IF	Achieved	Scores, flags, percentiles and latency recorded.
Incorporate drift detection and ensemble scoring	Achieved	ADWIN adaptation and 2-of-3 consensus observed.
Store, visualise and evaluate outputs	Achieved locally	SQLite persistence and Grafana dashboards demonstrated.

Table 4. Evaluation against project objectives.

CHAPTER 6 - CONCLUSION

6.1 Summary of Project Achievements

The project built an end-to-end Bitcoin anomaly-detection pipeline using Bitcoin Core, ZeroMQ/RPC, Kafka, three unsupervised detectors, ADWIN, Mini-Batch K-Means, SQLite and Grafana. At prototype scale, the research question is answered positively: continuous anomaly analysis can be supported by separating ingestion from model processing and retaining inspectable evidence.

6.2 Achievement of Objectives

The objectives were met by implementing the streaming pipeline, feature engineering, HST, Online Isolation Forest, Batch Isolation Forest, two-out-of-three consensus, drift monitoring, behavioural grouping and visual storage. Disagreement, latency, adaptation and explanations were retained so unusual transactions were not presented as automatically confirmed fraud.

6.3 Contribution and Practical Value

The contribution is the integration and evaluation of established techniques. Pre-confirmation capture, complementary unsupervised models, drift control, behavioural context, explainability, persistence and monitoring are combined so an investigator can inspect transaction details, model agreement, risk, drift, latency and explanation from one evidence base.

6.4 Limitations

Reliable fraud labels were unavailable, so fraud-classification accuracy cannot be claimed. The deployment was local rather than multi-broker, so horizontal scalability was not demonstrated. SQLite may become a bottleneck under heavier concurrent workloads, and the Bitcoin Core evidence showed incomplete synchronisation. Online and Batch Isolation Forest also produced occasional latency spikes. Hadoop, Tableau, Superset and Streamlit were considered but not retained in the live monitoring path. Mini-Batch K-Means was preferred to repeatedly running full K-Means because incremental updates better fit streaming.

6.5 Future Prospects

1. Repeat live capture with a fully synchronised Bitcoin node (Sections 4.1 and 6.4).
2. Test multi-broker Kafka under heavier load, measuring throughput, lag, recovery and model latency (Sections 3.2, 4.7 and 5.8).
3. Benchmark SQLite against a production database such as PostgreSQL (Sections 3.6, 3.8 and 6.4).
4. Add investigator-verified or labelled cases for precision and false-positive analysis (Sections 2.3, 5.7 and 6.4).
5. Run longer drift tests, richer wallet/graph features and structured analyst evaluation (Sections 4.4, 4.5 and 5.2-5.6).

CHAPTER 7 - REFERENCES

Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernandez-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., & Whittle, S. (2015). The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. Proceedings of the VLDB Endowment, 8(12), 1792-1803.

Baena-Garcia, M., del Campo-Avila, J., Fidalgo-Merino, R., Bifet, A., Gavalda, R., & Morales-Bueno, R. (2006). Early Drift Detection Method. Fourth International Workshop on Knowledge Discovery from Data Streams.

Bifet, A., & Gavalda, R. (2007). Learning from time-changing data with adaptive windowing. Proceedings of the 2007 SIAM International Conference on Data Mining, 443-448. https://doi.org/10.1137/1.9781611972771.42

Bitcoin Core. (2026). Block and transaction broadcasting with ZeroMQ. GitHub. https://github.com/bitcoin/bitcoin/blob/master/doc/zmq.md

Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, 93-104. https://doi.org/10.1145/335191.335388

Carletti, M., Terzi, M., & Susto, G. A. (2023). Interpretable anomaly detection with DIFFI: Depth-based feature importance of Isolation Forest. Engineering Applications of Artificial Intelligence, 119, 105730. https://doi.org/10.1016/j.engappai.2022.105730

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), Article 15. https://doi.org/10.1145/1541880.1541882

Cholevas, C., Angeli, E., Sereti, Z., Mavrikos, E., & Tsekouras, G. E. (2024). Anomaly detection in blockchain networks using unsupervised learning: A survey. Algorithms, 17(5), 201. https://doi.org/10.3390/a17050201

Ding, Z., & Fei, M. (2013). An anomaly detection approach based on Isolation Forest algorithm for streaming data using sliding window. IFAC Proceedings Volumes, 46(20), 12-17. https://doi.org/10.3182/20130902-3-CN-3020.00044

Gama, J., Medas, P., Castillo, G., & Rodrigues, P. P. (2004). Learning with drift detection. Advances in Artificial Intelligence - SBIA 2004, 286-295. https://doi.org/10.1007/978-3-540-28645-5_29

Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), Article 44. https://doi.org/10.1145/2523813

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. Proceedings of the NetDB Workshop 2011.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining, 413-422. https://doi.org/10.1109/ICDM.2008.17

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1, 281-297.

Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. https://bitcoin.org/bitcoin.pdf

Pham, T., & Lee, S. (2016). Anomaly detection in Bitcoin network using unsupervised learning methods. arXiv:1611.03941. https://doi.org/10.48550/arXiv.1611.03941

Raptis, T. P., & Passarella, A. (2022). On efficiently partitioning a topic in Apache Kafka. arXiv:2205.09415. https://doi.org/10.48550/arXiv.2205.09415

Sculley, D. (2010). Web-scale k-means clustering. Proceedings of the 19th International Conference on World Wide Web, 1177-1178. https://doi.org/10.1145/1772690.1772862

Shevchuk, R., Martsenyuk, V., Adamyk, B., Benson, V., & Melnyk, A. (2025). Anomaly detection in blockchain: A systematic review of trends, challenges, and future directions. Applied Sciences, 15(15), 8330. https://doi.org/10.3390/app15158330

Stojanovic, B., & Bozic, J. (2022). Robust financial fraud alerting system based in the cloud environment. Sensors, 22(23), 9461. https://doi.org/10.3390/s22239461

Tan, S. C., Ting, K. M., & Liu, F. T. (2011). Fast anomaly detection for streaming data. Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence, 1511-1516. https://doi.org/10.5591/978-1-57735-516-8/IJCAI11-254

Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., & Leiserson, C. E. (2019). Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. arXiv:1908.02591. https://doi.org/10.48550/arXiv.1908.02591

Zimek, A., Campello, R. J. G. B., & Sander, J. (2014). Ensembles for unsupervised outlier detection: Challenges and research questions. ACM SIGKDD Explorations Newsletter, 15(1), 11-22. https://doi.org/10.1145/2594473.2594476

CHAPTER 8 - APPENDICES

Appendix A - AI Declaration

AI was used for structuring as permitted by assessment at Level2. The author reviewed, edited and accepts responsibility for the submitted work.

Appendix B - Ethics Form (UREC 2)

The human evaluation used the University UREC 2 ethics route referenced in Sections 3.10 and 5.4. The approved form forms part of the project evidence package.



UREC2+Low+Risk+Human+Participants+2025-26v1 (1).pdf (Command Line)

Appendix C - Publication Procedure Form

The publication procedure documentation forms part of the assessment evidence package.

Sheffield Hallam University | College of Business, Technology and Engineering

Dissertation for Computing (55-708541).

PUBLICATION PROCEDURE FORM

In this module, while you create your own research question or topic area, your supervisor makes a significant intellectual contribution to this work as the research progresses. Your supervisor will make the decision on whether your work merits publication based on the quality of the work you have produced. Your supervisor will co-author the paper for publication with you and your supervisor will both be listed as authors. You are required to sign the declaration below to confirm that you understand and will follow this procedure.

Declaration:

I Joseph Aragbaiye confirm that I understand will comply with the Publication Procedure outlined in the Module Handbook and the Blackboard Site.		
Student:	Signature Joseph Aragbaiye	Date 16/09/2026
Supervisor:	Signature	Date

Appendix D - Participant Information Sheet

The participant information sheet referenced in Sections 5.2 and 5.4 forms part of the evaluation evidence package.

Sheffield Hallam University

PARTICIPANT INFORMATION SHEET

The following issues should be addressed where relevant. This could be, but does not have to be, in a question-answer format. Use a format that best meets the needs of your research participants, but the list below summarises the areas that need to be addressed to ensure participants are appropriately informed. Plain, easily understood language should be used with a minimum of technical or academic terms or jargon. Pay special attention to preparing material for children or adults with limited cognitive capacity - further information on this can be obtained from www.hra-decisiontools.org.uk/consent/

List of Contents Required: They do **not** have to be presented in this order as numbered questions; rather, this list provides a checklist of the material now required to be included in a Participant Information Sheet or equivalent briefing under GDPR.

Title of Project: Real-Time Monitoring Anomaly Detection in Bitcoin Transactions Using Big Data Streaming Tools, Machine Learning and Dashboard

Legal basis for research studies:
The University undertakes research as part of its function for the community under its legal status. Data protection allows us to use personal data for research with appropriate safeguards in place under the legal basis of **public tasks that are in the public interest.** A full statement of your rights can be found at: www.shu.ac.uk/about-this-website/privacy-policy/privacy-notices/privacy-notice-for-research. However, all University research is reviewed to ensure that participants are treated appropriately and their rights respected. This study was approved by the University's Research Ethics Committee with reference number *xxxxxxx*. Further information at: www.shu.ac.uk/research/excellence/ethics-and-integrity

1. **Opening statement:**

   This research focuses on the development and evaluation of a scalable, real-time anomaly detection system for Bitcoin transactions using Big Data streaming analytics and hybrid unsupervised machine learning techniques.

2. **Why have you asked me to take part?**

   You have been invited to participate because your expertise aligns with the scope of artificial intelligence and cyber security.

3. **Do I have to take part?**

   It is up to you to decide if you want to take part. A copy of the information provided here is yours to keep, along with the consent form if you do decide to take part. You can still decide to withdraw at any time without giving a reason, or you can decide not to answer a particular question.

4. **What will I be required to do?**

Appendix E - Anonymised Participant Consent Forms

Anonymised consent evidence referenced in Section 5.4 forms part of the evaluation evidence package.

University

PARTICIPANT CONSENT FORM

TITLE OF RESEARCH STUDY: Real-Time Monitoring Anomaly Detection in Bitcoin Transactions Using Big Data Streaming Tools, Machine Learning and Dashboard

Please answer the following questions by ticking the response that applies

	YES	NO
1. I have read the Information Sheet for this study and have had details of the study explained to me.	☒	☐
2. My questions about the study have been answered to my satisfaction and I understand that I may ask further questions at any point.	☒	☐
3. I understand that I am free to withdraw from the study within the time limits outlined in the Information Sheet, without giving a reason for my withdrawal or to decline to answer any particular questions in the study without any consequences to my future treatment by the researcher.	☒	☐
4. I agree to provide information to the researchers under the conditions of confidentiality set out in the Information Sheet.	☒	☐
5. I wish to participate in the study under the conditions set out in the Information Sheet.	☒	☐
6. I consent to the information collected for the purposes of this research study, once anonymised (so that I cannot be identified), to be used for any other research purposes.	☒	☐

Participant's Signature: _____J.Musa_____ Date: _____

Participant's Name (Printed): _____Joseph Musa_____

Contact details:
_____jmusa@niitfortesoft.com_____

Researcher's Name (Printed): _____Joseph Aragbaye_____

Researcher's Signature: _____ PMJ_____

Researcher's contact details:

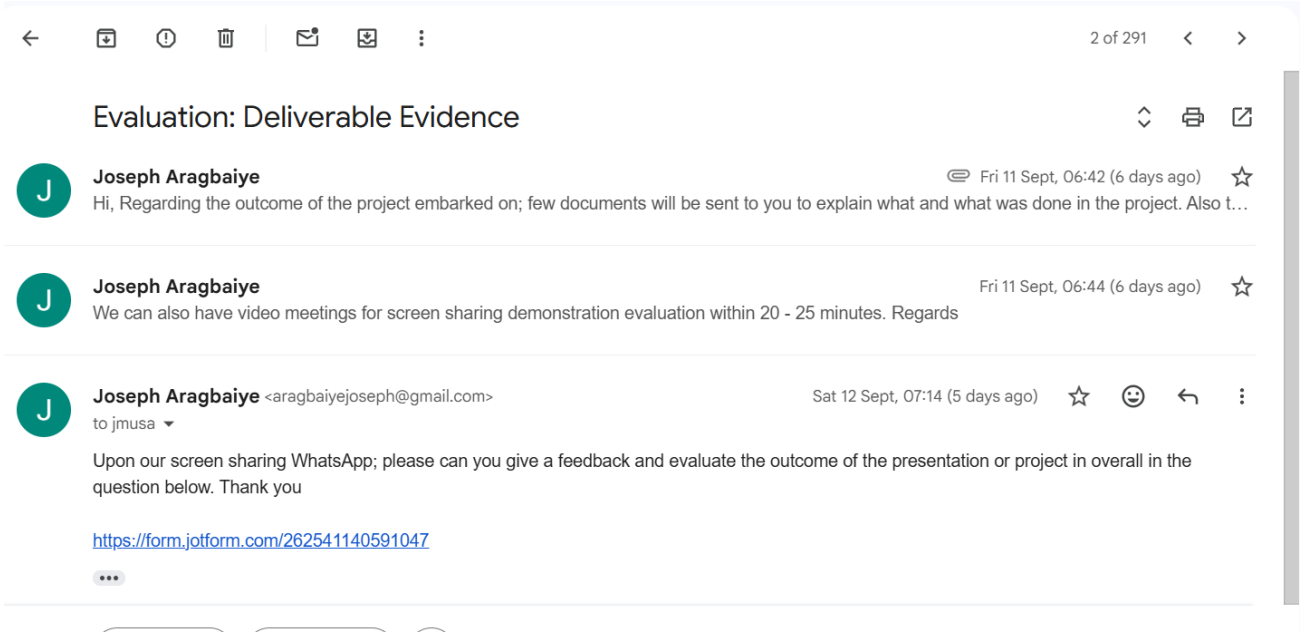
Name: Joseph Aragbaiye
Address: 17 Portobello House, Portobello Street, S14AA.

SAMPLE

Appendix F - Data Gathering / Evaluation Instruments

The evaluation instrument referenced in Sections 5.2-5.6 forms part of the evaluation evidence package.

Musa-Joseph-09-12-2026.pdf



https://www.jotform.com/pdf-editor/262541140591047?st=b3E0dkFVdmlUREhyTDFScm5pYTFMRDE0cXU2djNiNVp1L1Z6cGpINHFyT01FOFBPZ2h4Zm83SEVqbFFmS3dUQ0ZiWkU1TVNUNmt2SzZDalAxNjVjYmZBc1VLSEM1MER4bks1MnFHVDU4bitSK3Z1Z3hJQXY3SFNLQjRQNUZkU3Q=

Appendix G - Primary/Secondary Data and Software Code

GitHub is the designated repository for the project code and configuration files, with the submission copy also provided through the University submission route. The following terminal captures document model parameters, feature transformations, warm-up and confidence settings, adaptive control, ensemble weighting, verdict explanation, Kafka consumption and the SQLite result schema. Credentials and secrets are excluded.

```
(kafka_venv) admp@Joseph:~$ grep -n -E \
'FEATURE_NAMES|GLOBAL_WARMUP_SIZE|HST_WINDOW_SIZE|HST_TREES|HST_HEIGHT|ONLINE_IF_WINDOW_SIZE|ONLINE_IF_TREES|ONLINE_IF_MAX_LEAF_SAMPL
ES|BATCH_IF_WINDOW|BATCH_RETRAIN_INTERVAL|BATCH_IF_TREES|BATCH_IF_CONTAMINATION|KMEANS_BATCH_SIZE|KMEANS_CLUSTERS' \
"$FILE"
77:FEATURE_NAMES = [
94:GLOBAL_WARMUP_SIZE = read_int_setting("BTC_GLOBAL_WARMUP_SIZE", 400)
95:INITIAL_BATCH_TRAINING_SIZE = GLOBAL_WARMUP_SIZE
96:MIN_SCORE_HISTORY = GLOBAL_WARMUP_SIZE
97:ONLINE_IF_MIN_SCORE_HISTORY = max(1, GLOBAL_WARMUP_SIZE - 1)
123:HST_WINDOW_SIZE = read_int_setting("BTC_HST_WINDOW_SIZE", 250)
124:HST_TREES = read_int_setting("BTC_HST_TREES", 30)
125:HST_HEIGHT = read_int_setting("BTC_HST_HEIGHT", 10)
132:ONLINE_IF_WARMUP_SIZE = GLOBAL_WARMUP_SIZE
133:ONLINE_IF_WINDOW_SIZE = read_int_setting("BTC_ONLINE_IF_WINDOW_SIZE", 2_048)
134:ONLINE_IF_TREES = read_int_setting("BTC_ONLINE_IF_TREES", 32)
135:ONLINE_IF_MAX_LEAF_SAMPLES = read_int_setting(
136:    "BTC_ONLINE_IF_MAX_LEAF_SAMPLES",
151:BATCH_IF_WINDOW = read_int_setting("BTC_BATCH_IF_WINDOW", 1_000)
152:BATCH_RETRAIN_INTERVAL = read_int_setting("BTC_BATCH_RETRAIN_INTERVAL", 1_000)
153:BATCH_IF_TREES = read_int_setting("BTC_BATCH_IF_TREES", 200)
154:BATCH_IF_CONTAMINATION = read_float_setting("BTC_BATCH_IF_CONTAMINATION", 0.02)
204:KMEANS_BATCH_SIZE = read_int_setting("BTC_KMEANS_BATCH_SIZE", 32)
205:KMEANS_CLUSTERS = read_int_setting("BTC_KMEANS_CLUSTERS", 4)
710:        window_size: int = ONLINE_IF_WINDOW_SIZE,
725:        schema_columns = [*FEATURE_NAMES, "target"]
734:            num_trees=ONLINE_IF_TREES,
735:            max_leaf_samples=ONLINE_IF_MAX_LEAF_SAMPLES,
738:            window_size=ONLINE_IF_WINDOW_SIZE,
802:            n_trees=HST_TREES,
803:            height=HST_HEIGHT,
804:            window_size=HST_WINDOW_SIZE,
827:            maxlen=BATCH_IF_WINDOW
840:            for feature in FEATURE_NAMES
```

Close

Appendix Figure G1. Terminal evidence of the five model features, 400-transaction warm-up, HST, Online Isolation Forest, Batch Isolation Forest and Mini-Batch K-Means configuration.

```
(kafka_venv) admp@Joseph:~$ sed -n '/def build_transaction_features/,/def classify_transaction_shape/p' "$FILE"
    def build_transaction_features(tx: dict[str, Any]) -> dict[str, Any]:
        """Turn one decoded Bitcoin transaction into the five features shared by the detectors."""

        raw_values = np.asarray(
            [
                clean_bitcoin_metric(tx.get("total_output_btc")),
                clean_bitcoin_metric(tx.get("vsize")),
                clean_bitcoin_metric(tx.get("input_count")),
                clean_bitcoin_metric(tx.get("output_count")),
            ],
            dtype=np.float64,
        )
        raw_values = np.nan_to_num(
            raw_values,
            nan=0.0,
            posinf=0.0,
            neginf=0.0,
        )
        raw_values = np.maximum(raw_values, 0.0)

        amount, vsize, inputs, outputs = raw_values
        output_input_ratio = (outputs + 1.0) / (inputs + 1.0)

        model_vector = np.concatenate(
            [
                np.log1p(raw_values),
                np.asarray([np.log(output_input_ratio)], dtype=np.float64),
            ]
        )
        scaled = {
            name: float(value)
            for name, value in zip(FEATURE_NAMES, model_vector, strict=True)
```

Close

Appendix Figure G2. Feature-engineering code showing raw Bitcoin transaction inputs, output-to-input ratio calculation and logarithmic model-vector transformation.

```
            nan=0.0,
            posinf=0.0,
            neginf=0.0,
        )
        raw_values = np.maximum(raw_values, 0.0)

        amount, vsize, inputs, outputs = raw_values
        output_input_ratio = (outputs + 1.0) / (inputs + 1.0)

        model_vector = np.concatenate(
            [
                np.log1p(raw_values),
                np.asarray([np.log(output_input_ratio)], dtype=np.float64),
            ]
        )
        scaled = {
            name: float(value)
            for name, value in zip(FEATURE_NAMES, model_vector, strict=True)
        }

        # River HST expects bounded values. NumPy maps the positive log features
        # with x/(1+x), and maps the signed ratio with a logistic function.
        hst_vector = np.empty_like(model_vector)
        hst_vector[:4] = model_vector[:4] / (1.0 + model_vector[:4])
        bounded_ratio = np.clip(model_vector[4], -60.0, 60.0)
        hst_vector[4] = 1.0 / (1.0 + np.exp(-bounded_ratio))
        hst_features = {
            name: float(value)
            for name, value in zip(FEATURE_NAMES, hst_vector, strict=True)
        }

        return {
```

Appendix Figure G3. HST-specific bounded transformation using x/(1+x) and a logistic mapping for the signed ratio feature.

```
# Operational review rule. This does not prove fraud.
POLICY_BTC_THRESHOLD = read_float_setting("BTC_POLICY_THRESHOLD", 50.0)

# Warm-up and history sizes
# The first 400 unique rows train/calibrate the models. Transaction 401 is the
# first transaction eligible for voting. At least two models must flag before
# the transaction is treated as a confirmed anomaly detection.
GLOBAL_WARMUP_SIZE = read_int_setting("BTC_GLOBAL_WARMUP_SIZE", 400)
INITIAL_BATCH_TRAINING_SIZE = GLOBAL_WARMUP_SIZE
MIN_SCORE_HISTORY = GLOBAL_WARMUP_SIZE
ONLINE_IF_MIN_SCORE_HISTORY = max(1, GLOBAL_WARMUP_SIZE - 1)
HST_SCORE_HISTORY_SIZE = read_int_setting("BTC_HST_SCORE_HISTORY_SIZE", 1_000)
ONLINE_IF_SCORE_HISTORY_SIZE = read_int_setting(
    "BTC_ONLINE_IF_SCORE_HISTORY_SIZE",
    1_000,
)
LATENCY_HISTORY_SIZE = read_int_setting("BTC_LATENCY_HISTORY_SIZE", 500)
DUPLICATE_CACHE_SIZE = read_int_setting("BTC_DUPLICATE_CACHE_SIZE", 50_000)
SQLITE_TIMEOUT_SECONDS = read_float_setting("BTC_SQLITE_TIMEOUT_SECONDS", 15.0)
SMTP_TIMEOUT_SECONDS = read_float_setting("BTC_SMTP_TIMEOUT_SECONDS", 10.0)
STRUCTURE_MIN_COUNT = read_int_setting("BTC_STRUCTURE_MIN_COUNT", 10)
STRUCTURE_DOMINANCE = read_float_setting("BTC_STRUCTURE_DOMINANCE", 3.0)
COINJOIN_MIN_COUNT = read_int_setting("BTC_COINJOIN_MIN_COUNT", 5)
COINJOIN_RATIO_LOW = read_float_setting("BTC_COINJOIN_RATIO_LOW", 0.5)
COINJOIN_RATIO_HIGH = read_float_setting("BTC_COINJOIN_RATIO_HIGH", 2.0)
THREE_MODEL_CONFIDENCE_BONUS = read_float_setting(
    "BTC_THREE_MODEL_CONFIDENCE_BONUS",
    0.10,
)
```

Appendix Figure G4. Warm-up, history, structural settings and three-model confidence-bonus configuration used by the detector.

```
    """Switch the percentile and risk thresholds as the live stream behaviour changes."""

    settings = {
        "CALIBRATING": (
            STABLE_ANOMALY_PERCENTILE,
            STABLE_MEDIUM_RISK,
            STABLE_CRITICAL_RISK,
        ),
        "MORE_SENSITIVE": (
            SENSITIVE_ANOMALY_PERCENTILE,
            SENSITIVE_MEDIUM_RISK,
            SENSITIVE_CRITICAL_RISK,
        ),
        "STABLE": (
            STABLE_ANOMALY_PERCENTILE,
            STABLE_MEDIUM_RISK,
            STABLE_CRITICAL_RISK,
        ),
        "DRIFT_ADAPTATION": (
            DRIFT_ANOMALY_PERCENTILE,
            DRIFT_MEDIUM_RISK,
            DRIFT_CRITICAL_RISK,
        ),
        "STRICT": (
            STRICT_ANOMALY_PERCENTILE,
            STRICT_MEDIUM_RISK,
            STRICT_CRITICAL_RISK,
        ),
    }
```

Appendix Figure G5. Adaptive-mode mapping connecting CALIBRATING, MORE_SENSITIVE, STABLE, DRIFT_ADAPTATION and STRICT states to anomaly and risk thresholds.

```
    )
    self.update_reliability_weights()
    self.last_adaptive_review_row = self.rows_seen

    if self.confirmed_detection_rate >= HIGH_CONFIRMED_DETECTION_TRIGGER:
        self.set_adaptive_mode(
            "STRICT",
            "Confirmed anomaly rate is unusually high; sensitivity was "
            "reduced to control false positives.",
        )
    elif (
        self.high_risk_unconfirmed_rate
        >= HIGH_RISK_UNCONFIRMED_TRIGGER
        or self.single_model_rate
        >= SINGLE_MODEL_DISAGREEMENT_TRIGGER
    ):
        self.set_adaptive_mode(
            "MORE_SENSITIVE",
            "Recent high-risk unconfirmed transactions or repeated "
            "single-model disagreement indicate that sensitivity should "
            "increase.",
        )
    else:
        self.set_adaptive_mode(
            "STABLE",
            "Recent detection and disagreement rates are within the "
            "expected operating range.",
        )
```

Appendix Figure G6. Adaptive-controller decision logic selecting STRICT, MORE_SENSITIVE or STABLE operation from recent detection and disagreement evidence.

```
# Equal weights are the starting point. The controller can later make small
# reliability-based changes, but no weight can create a detection by itself.
MODEL_WEIGHT = 1.0 / 3.0
MIN_MODEL_WEIGHT = read_float_setting("BTC_MIN_MODEL_WEIGHT", 0.20)
MAX_MODEL_WEIGHT = read_float_setting("BTC_MAX_MODEL_WEIGHT", 0.50)
```

Appendix Figure G7. Initial equal ensemble weighting and permitted reliability-weight bounds for HST, Online Isolation Forest and Batch Isolation Forest.

```
        anomaly_votes = sum([hst_flag, online_if_flag, batch_if_flag])
        if verdict == "CRITICAL":
            decision = (
                f"Critical: {anomaly_votes} models flagged the transaction; "
                f"combined risk={risk_score:.4f}."
            )
        elif verdict == "MEDIUM_RISK":
            decision = (
                f"Medium risk: two models flagged the transaction; "
                f"combined risk={risk_score:.4f}."
            )
        elif verdict == "LESS_CRITICAL":
            decision = (
                f"Less critical: two models flagged the transaction, but "
                f"combined risk={risk_score:.4f} stayed below the medium "
                "threshold."
            )
        elif verdict == "MODEL_DISAGREEMENT":
            decision = (
                "Model disagreement: only one model flagged the transaction. "
                "At least two model flags are required for detection."
            )
        elif verdict == "CALIBRATING":
            decision = (
                f"Calibrating: the first {GLOBAL_WARMUP_SIZE} transactions "
                "are used for model learning before verdict voting begins."
            )
        else:
            decision = (
```

Appendix Figure G8. Human-readable verdict explanation showing anomaly-vote interpretation, two-model confirmation and calibration behaviour.

```
# Kafka consumer entry point

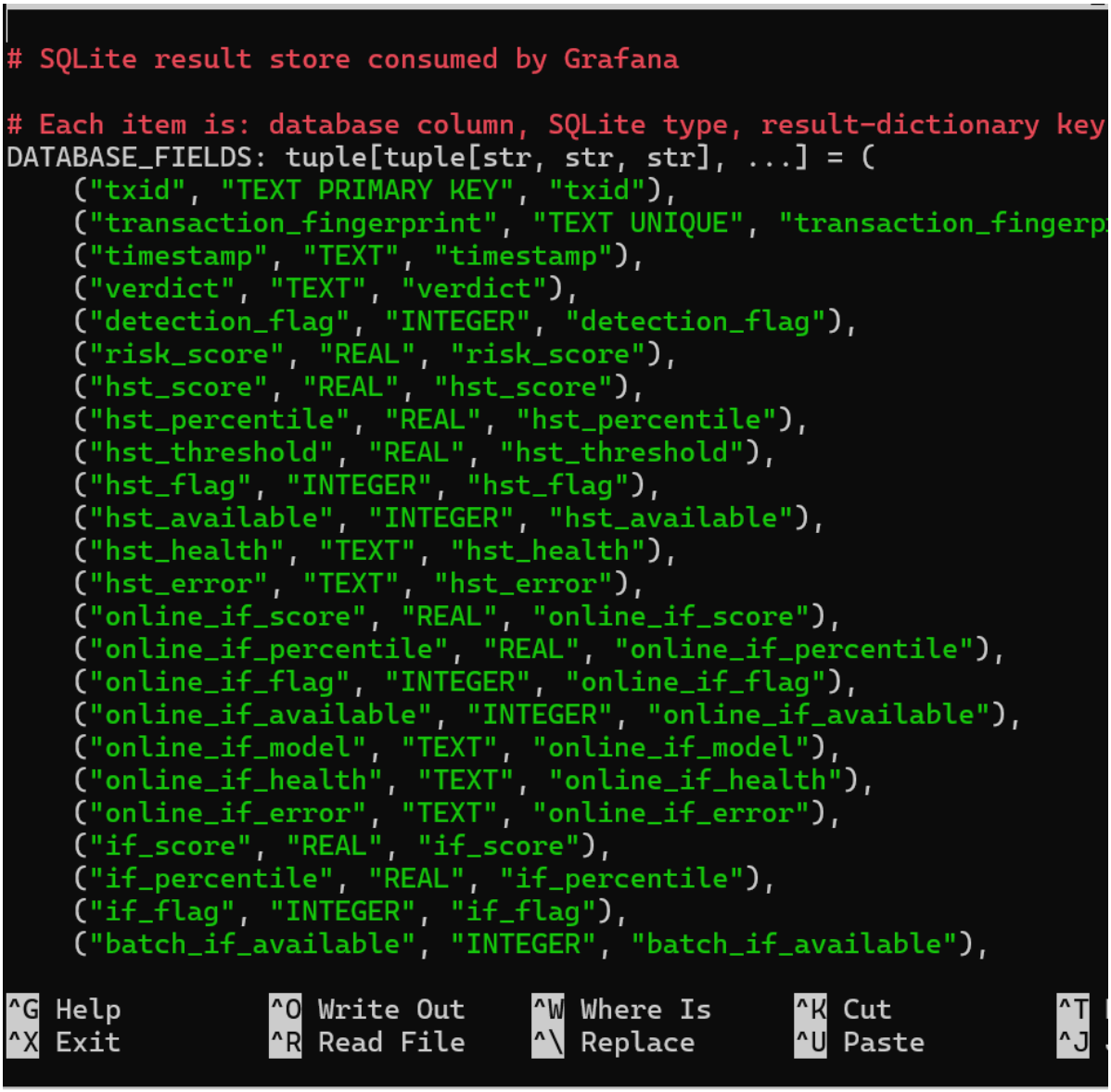

def run_bitcoin_anomaly_pipeline() -> None:
    """Consume btc-tx messages, reject duplicates, score them, and persist each finished verdict."""

    prepare_results_database()
    engine = BitcoinAnomalyEngine()
    duplicate_guard = TransactionDuplicateGuard()

    consumer = KafkaConsumer(
        KAFKA_TOPIC,
        bootstrap_servers=KAFKA_SERVERS,
        value_deserializer=lambda raw: json.loads(raw.decode("utf-8")),
        auto_offset_reset="latest",
        enable_auto_commit=True,
        group_id=KAFKA_GROUP_ID,
    )

    print(
        "[*] Bitcoin anomaly pipeline using CapyMOA Online-iForest "
        f"and listening to Kafka topic '{KAFKA_TOPIC}' on {KAFKA_SERVERS}."
    )
```

Appendix Figure G9. Kafka consumer entry point for topic btc-tx, including duplicate protection, model-engine initialisation and JSON deserialisation.



Appendix Figure G10. SQLite result-store schema exposing identifiers, verdict, risk, scores, percentiles, flags, availability and health fields for Grafana.

Appendix H - Research Project Plan

The research project plan is referenced in Section 3.11 and is retained with the project planning evidence.

Project Title

Real-Time Monitoring Anomaly Detection in Bitcoin Transactions Using Big Data Streaming Tools, Machine Learning and Dashboard

Project Aim

The aim of this project was to design, implement and evaluate a real-time system for identifying unusual patterns in Bitcoin transaction streams using streaming technologies and machine-learning techniques.

Original Project Objectives

1. To develop a real-time pipeline for processing Bitcoin transaction streams using the Mempool.space API, Apache Kafka and Spark Structured Streaming.
2. To detect anomalous Bitcoin transactions using Isolation Forest, K-Means clustering and related analytical techniques.

3. To analyse transaction relationships and behavioural patterns in real time using streaming and graph-based analysis.
4. To generate structured outputs using HDFS and Hive, provide visualisation through Tableau, and generate automated SMTP email alerts for suspicious transaction activity.
5. To evaluate the scalability and performance of the system when processing high-velocity Bitcoin transaction streams.

Original Proposed Architecture

The project was originally planned around the following implementation structure:

Bitcoin Transactions

Mempool.space API

Apache Kafka

Spark Structured Streaming

Isolation Forest

K-Means Clustering

Graph-Based Analysis

HDFS and Hive

Tableau

SMTP Email Alerts

Planned Project Activities

The project was divided into several planned stages.

April 2026

The project topic was defined and the initial research problem, aim and objectives were established.

The project planning process also included the identification of suitable technologies, research areas and expected project outputs.

April to May 2026

A literature review was carried out to examine previous research relating to Bitcoin anomaly detection, cryptocurrency transaction analysis, real-time data processing and machine learning.

The original research proposal and system requirements were also developed during this period.

May to June 2026

The proposed system architecture was designed.

The original technology stack included the Mempool.space API, Apache Kafka, Spark Structured Streaming, Isolation Forest, K-Means, GraphFrames, HDFS, Hive and Tableau.

June to July 2026

The real-time transaction-processing pipeline was planned for implementation.

This stage focused on transaction ingestion, streaming, data preparation and feature engineering.

July to August 2026

The machine-learning and anomaly-detection components were planned for development and testing.

This included the implementation of anomaly-detection methods and behavioural analysis techniques.

August 2026

Storage, visualisation and alerting components were planned for integration.

The original design included HDFS and Hive for storage, Tableau for visualisation and SMTP email notification for alerts.

August to September 2026

The system was planned for evaluation.

The evaluation focused on system operation, processing performance, anomaly-detection behaviour, scalability and the effectiveness of the monitoring process.

The final report, limitations, discussion and project documentation were also completed during this period.

September 2026

The project was completed and prepared for final submission.

Changes During Implementation

During implementation, the original project architecture was revised after further technical investigation and practical testing.

The Mempool.space API was replaced by Bitcoin Core because Bitcoin Core provided direct access to live Bitcoin transaction information and greater control over transaction ingestion.

ZeroMQ and RPC were introduced to obtain live transaction notifications and retrieve additional transaction information directly from the Bitcoin node.

Apache Kafka was retained as the streaming layer because it provided buffering and separation between transaction ingestion and downstream processing.

Spark Structured Streaming was replaced by a Python-based processing and machine-learning pipeline. This provided greater control over online anomaly scoring, model state, adaptive processing and concept-drift handling.

The original machine-learning approach was also refined. The final system compared three unsupervised anomaly-detection models:

Half-Space Trees

Online Isolation Forest

Batch Isolation Forest

Additional components were introduced to support:

ADWIN concept-drift detection

Model consensus

Behavioural clustering

Explainability

The original HDFS and Hive storage approach was replaced by SQLite because a lightweight evidence store was more suitable for the implemented single-node environment.

Tableau was replaced by Grafana for live operational monitoring and dashboard visualisation.

Final Implemented Architecture

The final system followed this structure:

Bitcoin Core

ZeroMQ and RPC

Apache Kafka

Python-Based Streaming and Machine-Learning Processing

Half-Space Trees

Online Isolation Forest

Batch Isolation Forest

ADWIN Concept-Drift Detection

Model Consensus

Behavioural Clustering

Explainability

SQLite

Grafana

Email Alerts

Relationship Between the Original Plan and Final Implementation

Although several technologies changed during implementation, the main purpose of the research remained unchanged.

The project continued to focus on the development and evaluation of a real-time Bitcoin transaction anomaly-detection pipeline.

The changes mainly affected the technical methods used to achieve the research objectives.

The final implementation provided direct Bitcoin transaction access, continuous streaming, adaptive anomaly detection, evidence storage, monitoring and alerting.

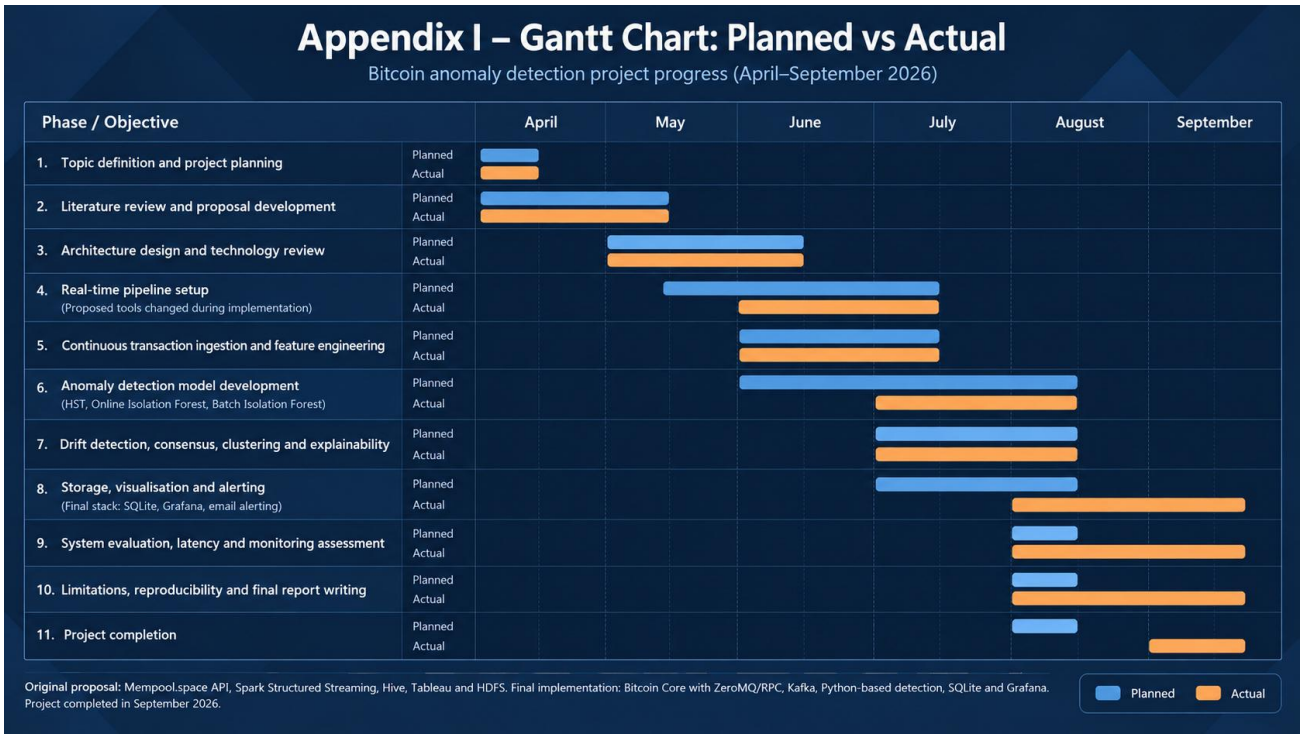
Detected anomalies were treated as indicators of unusual behaviour requiring further investigation and not as confirmed evidence of fraud.

Appendix Statement

This appendix presents the original research project plan, including the project aim, objectives, proposed architecture, implementation stages and planned schedule. It provides the planning baseline against which subsequent technical changes, implementation decisions and actual project progress were assessed.

Appendix I - Gantt Chart: Planned vs Actual

The planned-versus-actual Gantt chart is referenced in Section 3.11 and retained with project planning evidence.



Appendix J - System Test Cases and Evidence

The detailed system-test evidence supporting Table 2 and Section 3.7 is retained in the project evidence package.

Recording 2026-09-17 120911.mp4

Appendix K - Risk Register and Risk Map

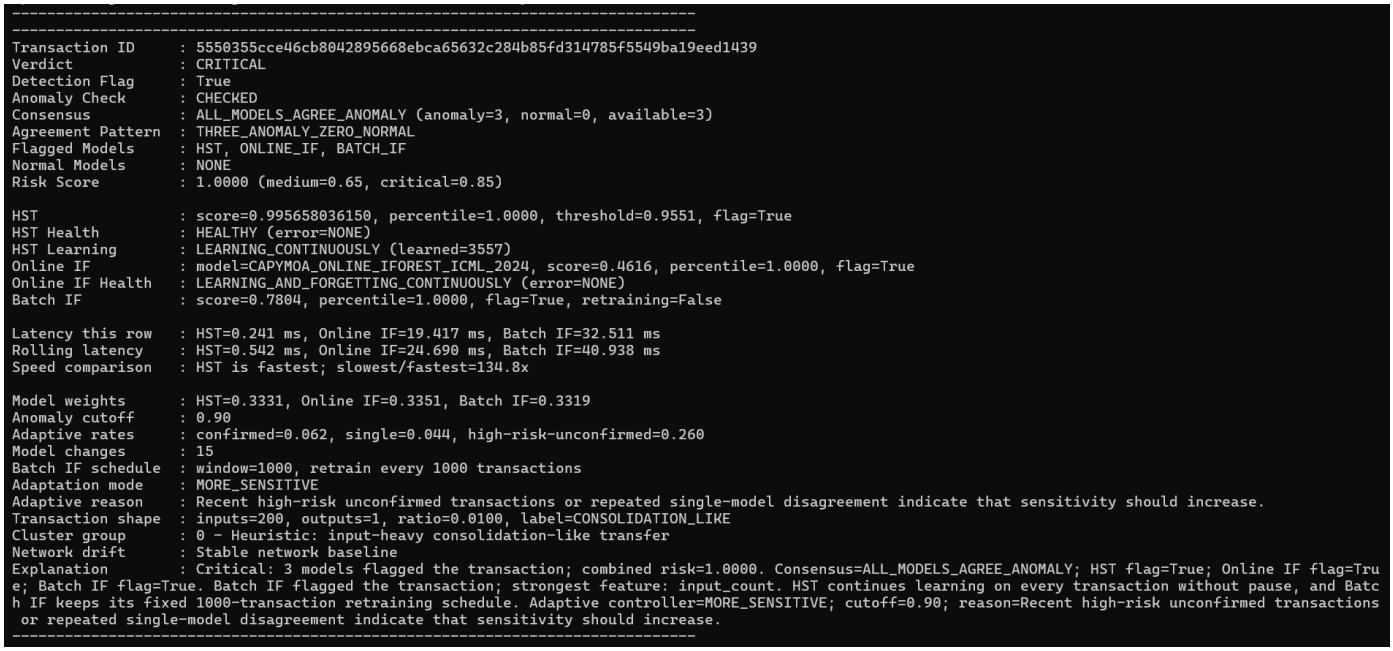
The full risk register and risk map supporting Section 3.8 are retained in the project evidence package.

Risk	Likelihood	Impact	Mitigation

Incomplete node synchronisation	High	High	Verify chain status; separate bootstrap/replay evidence from fully live evidence.
Kafka unavailable / lag	Medium	High	Monitor broker/consumer health, buffer safely and restart without silent loss.
SQLite locking	Medium	Medium	Use short, controlled writes and retries; consider production database at scale.
Concept drift / false alerts	High	High	Use ADWIN state, conservative adaptation and model consensus.
Latency spikes	Medium	Medium	Track per-model latency and bound expensive retraining/scoring work.
Data loss / malformed records	Low	High	Validate records, retain identifiers and log failures for replay/audit.

Appendix L - Implementation Evidence

Additional implementation screenshots and terminal evidence supporting Section 4.1 are retained in the project evidence package.



```
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
Transaction ID     : 5550355cce46cb8042895668ebca65632c284b85fd314785f5549ba19eed1439
Verdict            : CRITICAL
Detection Flag     : True
Anomaly Check      : CHECKED
Consensus          : ALL_MODELS_AGREE_ANOMALY (anomaly=3, normal=0, available=3)
Agreement Pattern  : THREE_ANOMALY_ZERO_NORMAL
Flagged Models     : HST, ONLINE_IF, BATCH_IF
Normal Models      : NONE
Risk Score         : 1.0000 (medium=0.65, critical=0.85)

HST                : score=0.995658036150, percentile=1.0000, threshold=0.9551, flag=True
HST Health         : HEALTHY (error=NONE)
HST Learning       : LEARNING_CONTINUOUSLY (learned=3557)
Online IF          : model=CAPYMOA_ONLINE_IFOREST_ICML_2024, score=0.4616, percentile=1.0000, flag=True
Online IF Health   : LEARNING_AND_FORGETTING_CONTINUOUSLY (error=NONE)
Batch IF           : score=0.7804, percentile=1.0000, flag=True, retraining=False

Latency this row   : HST=0.241 ms, Online IF=19.417 ms, Batch IF=32.511 ms
Rolling latency    : HST=0.542 ms, Online IF=24.690 ms, Batch IF=40.938 ms
Speed comparison   : HST is fastest; slowest/fastest=134.8x

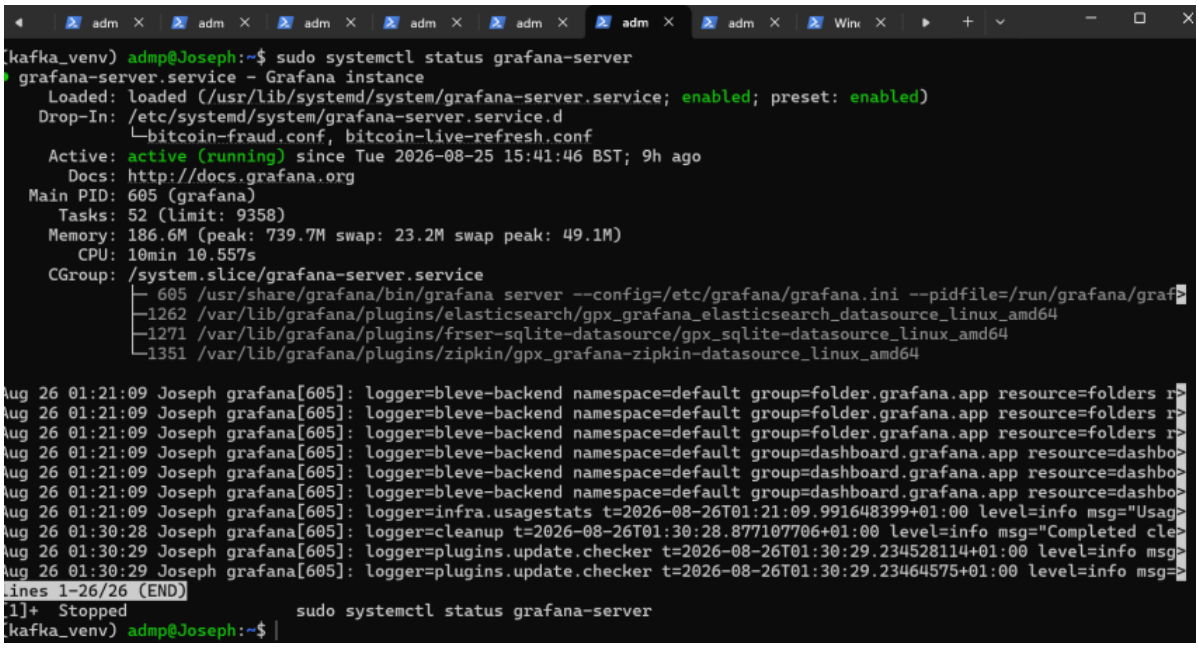
Model weights      : HST=0.3331, Online IF=0.3351, Batch IF=0.3319
Anomaly cutoff     : 0.90
Adaptive rates     : confirmed=0.062, single=0.044, high-risk-unconfirmed=0.260
Model changes      : 15
Batch IF schedule  : window=1000, retrain every 1000 transactions
Adaptation mode    : MORE_SENSITIVE
Adaptive reason    : Recent high-risk unconfirmed transactions or repeated single-model disagreement indicate that sensitivity should increase.
Transaction shape  : inputs=200, outputs=1, ratio=0.0100, label=CONSOLIDATION_LIKE
Cluster group      : 0 - Heuristic: input-heavy consolidation-like transfer
Network drift      : Stable network baseline
Explanation        : Critical: 3 models flagged the transaction; combined risk=1.0000. Consensus=ALL_MODELS_AGREE_ANOMALY; HST flag=True; Online IF flag=Tru
e; Batch IF flag=True. Batch IF flagged the transaction; strongest feature: input_count. HST continues learning on every transaction without pause, and Batc
h IF keeps its fixed 1000-transaction retraining schedule. Adaptive controller=MORE_SENSITIVE; cutoff=0.90; reason=Recent high-risk unconfirmed transactions
 or repeated single-model disagreement indicate that sensitivity should increase.
--------------------------------------------------------------------------------
```

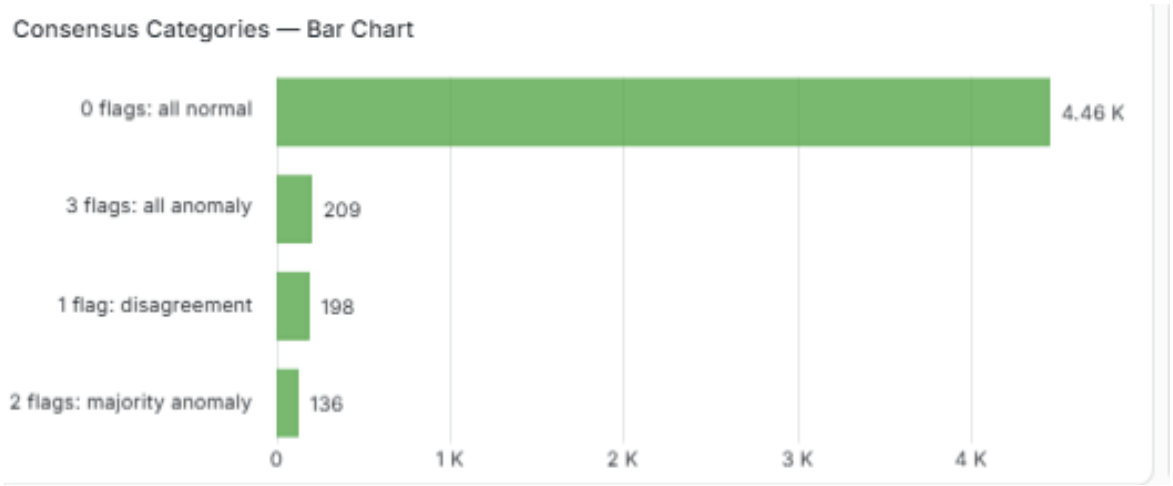
```
a799b5c26312e5","version":1,"size":194,"vsize":113,"weight":449,"locktime":0,"input_txids":["b8a996d17487b148690a78a5effeefd784194704
379e2d699afe3c09c5e42b57"],"input_count":1,"receivers":[{"address":"19cTaaTLf8gf6dqKqDws21RZ3qwC4wND49","btc":0.00062802}],"output_co
unt":1,"total_output_btc":0.00062802,"fee_btc":null,"year":"2026","month":"09","time_recorded":"08:28:57","timestamp":"2026-09-17T08:
28:57.398611+00:00","block_height":936370,"blockchain_mediantime":"2026-02-13T10:53:48+00:00"}
{"txid":"72d35405b723d58573f3e5f7b38389403a97cbab1fba0020faad410bb6609b16","hash":"eab23b3a08ce2caa40cf35c4c2f9bb62ea0d9be155fcedad740
b93cec3e7add2","version":1,"size":223,"vsize":141,"weight":562,"locktime":0,"input_txids":["f0bc6f9c81692762ee7e6e9b30e55ce8a1398585
6900f4db34d3d365cf7052a7"],"input_count":1,"receivers":[{"address":"bc1q2a6er57qe6w5mv96u7hv2eqj97yzyfy35rvkyy","btc":0.00012813},{"a
ddress":"bc1qsuxf0c7h6dm5lmu396srn50chs2ve3v5r4ma39","btc":0.00047847}],"output_count":2,"total_output_btc":0.0006066,"fee_btc":null,
"year":"2026","month":"09","time_recorded":"08:28:57","timestamp":"2026-09-17T08:28:57.403562+00:00","block_height":936370,"blockchai
n_mediantime":"2026-02-13T10:53:48+00:00"}
{"txid":"e1c57d8784293ce34bb49b23426b1f849092fcf58bf929b9b51a18916f2be017","hash":"948d909561f509c342da7b35989e2f2ff61c4bf81c8c4d5600
c86bbe16e7d430","version":1,"size":195,"vsize":113,"weight":450,"locktime":0,"input_txids":["7f8d6051e50df6018ec6780fece2c9a917d037f1
a29a0124c4287117e27d0472"],"input_count":1,"receivers":[{"address":"1DcqYQqJSAakL46GczHGAnzQSHXTM2iuFM","btc":0.00422397}],"output_co
unt":1,"total_output_btc":0.00422397,"fee_btc":null,"year":"2026","month":"09","time_recorded":"08:28:57","timestamp":"2026-09-17T08:
28:57.426886+00:00","block_height":936370,"blockchain_mediantime":"2026-02-13T10:53:48+00:00"}
{"txid":"5b655a1f5e8e643131cfc1ae135e74c9d8ef8b30411420aa2498c44db723f018","hash":"9a0eb100faf75c0cc41045adb44b962982286b883bc1a39b1c
8183866bd15458","version":1,"size":222,"vsize":141,"weight":561,"locktime":0,"input_txids":["2aa6d1a3c242431cdf2f65eafe415d84461b3afe
0da98857083bbd97c16e58c5"],"input_count":1,"receivers":[{"address":"bc1q7ep7d5zdx05g9rdtaetjq4y7fzxhrk6zphlp42","btc":0.00107638},{"a
ddress":"bc1q7fpvhm5vtxj28j0xfc754wq5s22u0fg26xpfje","btc":1.221e-05}],"output_count":2,"total_output_btc":0.00108859,"fee_btc":null,
"year":"2026","month":"09","time_recorded":"08:28:57","timestamp":"2026-09-17T08:28:57.442690+00:00","block_height":936370,"blockchai
n_mediantime":"2026-02-13T10:53:48+00:00"}
{"txid":"7b38cd380de5e314acd4a03847b3a88e69633c4875a9507db1326836c00ecd19","hash":"409f3ec685a7d88b035845777c19ba6c0911fd4cd7fcfa7e5d
accad64874f299","version":2,"size":192,"vsize":111,"weight":441,"locktime":934526,"input_txids":["6c3b2df8a314934421b6ba154787343bd5b
bdf38e79051b2c8687088ba37500b"],"input_count":1,"receivers":[{"address":"3G2idutCzCQAahMyDH72wqnQBBA3JwqTYu","btc":0.02830486}],"outp
ut_count":1,"total_output_btc":0.02830486,"fee_btc":null,"year":"2026","month":"09","time_recorded":"08:28:57","timestamp":"2026-09-1
7T08:28:57.438781+00:00","block_height":936370,"blockchain_mediantime":"2026-02-13T10:53:48+00:00"}
{"txid":"f4f36a95d8998085d3e46c7c54988719b93960e9b2a2591b57f01ac5f523b31a","hash":"a5977695fb44a77cc5282650357ea93a23d13a647b0f0a39d2
fd78bff7a091ae","version":2,"size":192,"vsize":110,"weight":438,"locktime":0,"input_txids":["57db0b5b3e9add1ab6ce36a75a5011fb65a611c8
e3cb9fffa77b143b73aeac0c"],"input_count":1,"receivers":[{"address":"bc1qwga9claq43yc52agxrnt4mkflwnwnrzjuh6yj9","btc":0.0003089}],"ou
tput_count":1,"total_output_btc":0.0003089,"fee_btc":null,"year":"2026","month":"09","time_recorded":"08:28:57","timestamp":"2026-09-
17T08:28:57.565357+00:00","block_height":936370,"blockchain_mediantime":"2026-02-13T10:53:48+00:00"}
{"event":"queue_full","txid":"71230968a4068a48e25fb73506d956f7a24b9c01602803f85e81487c7c067fa7","queue_size":50000}
```


```
admp@Joseph:~$ sqlite3 -header -column /home/admp/bitcoin-fraud-data/bitcoin_analytics.db "SELECT timestamp,substr(txid,1,16
) AS txid,verdict,risk_score,anomaly_votes FROM transaction_verdicts ORDER BY rowid DESC LIMIT 1000;"
timestamp                 txid              verdict             risk_score          anomaly_votes
------------------------  ----------------  ------------------  ------------------  -------------
2025-03-17T23:32:58+00:00  a0a737d5128b109e  NORMAL              0.1583622           0
2025-03-17T23:32:58+00:00  97c3163d2ac80dbd  NORMAL              0.1243698           0
2025-03-17T23:32:58+00:00  c910029eb6eba5f5  NORMAL              0.1390576           0
2025-03-17T23:32:58+00:00  f4f35ad9c747b8cb  NORMAL              0.1403908           0
2025-03-17T23:32:58+00:00  fbe98a55e9ab0a8c  NORMAL              0.1656434           0
2025-03-17T22:39:38+00:00  21e4e5847f4c7142  NORMAL              0.1757164           0
2025-03-17T22:39:38+00:00  4194d004317be565  NORMAL              0.110044            0
2025-03-17T22:39:38+00:00  7246564f3ad4054f  NORMAL              0.1049318           0
2025-03-17T21:51:20+00:00  98dab8202cd23065  NORMAL              0.1204498           0
2025-03-17T21:51:20+00:00  cce52af6303baa80  NORMAL              0.1191634           0
2025-03-17T21:04:16+00:00  9b8d8b22fcfe5acf  NORMAL              0.1176812           0
2025-03-17T21:04:16+00:00  23aa132ef088e89d  NORMAL              0.123802            0
2025-03-17T21:04:16+00:00  331853624faf7609  NORMAL              0.1467368           0
2025-03-17T20:27:07+00:00  19dcb7774ac305d8  NORMAL              0.1274554           0
2025-03-17T20:27:07+00:00  ec6449343e796d30  NORMAL              0.1484132           0
2025-03-17T20:27:07+00:00  b0d476d6ed0b0aed  NORMAL              0.1324162           0
2025-03-17T20:27:07+00:00  ae302d9165324564  NORMAL              0.2716392           0
2025-03-17T19:36:41+00:00  6949cf91cce30333  NORMAL              0.2791414           0
2025-03-17T19:36:41+00:00  f179a14bc0c225ad  NORMAL              0.1471404           0
2025-03-17T19:36:41+00:00  87dc5f915465c653  NORMAL              0.2100332           0
2025-03-17T19:17:58+00:00  f56309fec56e268c  NORMAL              0.1245534           0
2025-03-17T19:17:58+00:00  088544b3b282f4b3  NORMAL              0.118967            0
2025-03-17T19:17:58+00:00  2119bd046fa39ac5  NORMAL              0.1344954           0
2025-03-17T18:51:12+00:00  64ede1419257f493  NORMAL              0.3911878           0
2025-03-17T18:51:12+00:00  2d3418704206862   NORMAL              0.141779            0
2025-03-17T18:51:12+00:00  7a5f3e0f56ede20b  NORMAL              0.1452516           0
2025-03-17T18:13:28+00:00  279cb2057a9b0ffb  NORMAL              0.3038532           0
```

2025-03-14T07:10:50+00:00	39ffb0840fc9f6c7	NORMAL	0.350802533333333	0
2025-03-14T07:10:50+00:00	0a6f0e43372f156b	NORMAL	0.1414436	0
2025-03-14T07:10:50+00:00	9a67d711be7ba133	NORMAL	0.424480533333333	0
2025-03-14T07:10:50+00:00	fe2f20e33453e3ea	NORMAL	0.343646666666667	0
2025-03-14T07:10:50+00:00	b5d92f75a7cf4b03	NORMAL	0.354344533333333	0
2025-03-14T07:10:50+00:00	33792a1523b51935	NORMAL	0.401943333333333	0
2025-03-14T07:10:50+00:00	4249070064ce0cac	CRITICAL	1.0	3
2025-03-14T07:10:50+00:00	56aa07e0616534b3	CRITICAL	0.9839864	2
2025-03-14T07:10:50+00:00	69ea880aba83eafa	CRITICAL	1.0	3
2025-03-14T07:10:50+00:00	570fdca4512fefb3	CRITICAL	0.983298933333333	2
2025-03-14T07:10:50+00:00	fd162a5107c9a8cf	CRITICAL	0.978633866666667	2
2025-03-14T07:10:50+00:00	f2b9d37a97bd42a9	CRITICAL	0.977129466666667	2
2025-03-14T07:10:50+00:00	9884589da33e9a74	MODEL_DISAGREEMENT	0.933660266666667	1
2025-03-14T07:10:50+00:00	d0fce3f056022652	NORMAL	0.897260533333333	0
2025-03-14T07:10:50+00:00	33c52b94b71eb70f	CRITICAL	0.967262933333333	2
2025-03-14T07:10:50+00:00	42807bd9cbd4a0c2	CRITICAL	0.967763333333333	2
2025-03-14T07:10:50+00:00	9d2b23b8f0f081e9	CRITICAL	0.968266133333333	2
2025-03-14T07:10:50+00:00	3e5be71dc92d31f6	NORMAL	0.882690666666667	0
2025-03-14T07:10:50+00:00	762a4fa2587145e1	NORMAL	0.898679466666667	0
2025-03-14T07:10:50+00:00	9384001079a277ea	NORMAL	0.842282266666667	0
2025-03-14T07:10:50+00:00	a7acd0ee21ee41df	NORMAL	0.881821733333333	0
2025-03-14T07:10:50+00:00	3f181d3c2311b4ba	NORMAL	0.8419616	0
2025-03-14T07:10:50+00:00	3556764b31aa492e	NORMAL	0.842462	0
2025-03-14T07:10:50+00:00	85cf231dabe305f8	NORMAL	0.514613333333333	0
2025-03-14T07:10:50+00:00	f7ed1d1b92a4b8f5	NORMAL	0.528814	0
2025-03-14T07:10:50+00:00	cace82e34e6d726e	NORMAL	0.539535866666667	0
2025-03-14T07:10:50+00:00	a1844b5536bae846	NORMAL	0.540037066666667	0
2025-03-14T07:10:50+00:00	4f35bea29a3502b7	NORMAL	0.761663466666667	0
2025-03-14T07:10:50+00:00	e886596b8ba8875f	NORMAL	0.7434708	0

```
admp@Joseph:~$ bitcoin-cli getnetworkinfo
{
  "version": 270000,
  "subversion": "/Satoshi:27.0.0/",
  "protocolversion": 70016,
  "localservices": "0000000000000c08",
  "localservicesnames": [
    "WITNESS",
    "NETWORK_LIMITED",
    "P2P_V2"
  ],
  "localrelay": true,
  "timeoffset": 0,
  "networkactive": true,
  "connections": 1,
  "connections_in": 0,
  "connections_out": 1,
  "networks": [
    {
      "name": "ipv4",
      "limited": false,
      "reachable": true,
      "proxy": "",
      "proxy_randomize_credentials": false
    },
    {
      "name": "ipv6",
      "limited": false,
      "reachable": true,
      "proxy": "",
```





Appendix M - Additional Graphs, Tables and Supporting Material

Additional supporting material is referenced where relevant in Chapters 3-5 and retained with the project evidence package.

Investigator Transaction Ledger

sequence	timestamp	txid	verdict	detection_flag	risk_score	anomaly_votes	normal_votes	available_models
598861	2026-08-26T03:53:2	58b9abc6f9f9c2aa6(	NORMAL	0	0.746	0	3	3
598860	2026-08-26T03:53:2	1d1feaebf95c17cabe8	NORMAL	0	0.330	0	3	3
598859	2026-08-26T03:53:2	fddd1be84135f85d05	CRITICAL	1	1	2	1	3
598858	2026-08-26T03:53:2	579323e35c249a81b	NORMAL	0	0.855	0	3	3
598857	2026-08-26T03:53:2	df72166005768b1282	CRITICAL	1	0.970	2	1	3
598856	2026-08-26T03:53:2	f44973d9c2f29f7001	NORMAL	0	0.772	0	3	3
598855	2026-08-26T03:53:2	d66a42b2632a1b7a0	CRITICAL	1	1	2	1	3
598854	2026-08-26T03:53:2	6f0b73f3d725603a11	NORMAL	0	0.435	0	3	3
598853	2026-08-26T03:53:2	03887d367919c2485	NORMAL	0	0.470	0	3	3
598852	2026-08-26T03:53:2	e97e36828f39c8b38	CRITICAL	1	1	2	1	3
598851	2026-08-26T03:53:2	da8336861f1c0d38d(	NORMAL	0	0.687	0	3	3

action Ledger

hst_score	hst_percentile	hst_threshold	hst_flag	hst_health	online_if_score	nline_if_percentile
0.731	0.742	0.933	0	HEALTHY	0.270	0.732
0.494	0.446	0.933	0	HEALTHY	0.237	0.315
0.972	0.978	0.932	1	HEALTHY	0.354	0.924
0.876	0.860	0.932	0	HEALTHY	0.291	0.842
0.944	0.944	0.930	1	HEALTHY	0.304	0.871
0.784	0.788	0.930	0	HEALTHY	0.270	0.738
0.967	0.973	0.929	1	HEALTHY	0.343	0.919
0.498	0.481	0.929	0	HEALTHY	0.251	0.553
0.498	0.481	0.929	0	HEALTHY	0.251	0.555
0.965	0.968	0.929	1	HEALTHY	0.362	0.928
0.773	0.778	0.929	0	HEALTHY	0.247	0.462

network_drift	adaptation_mode	hst_ms	online_if_ms	batch_if_ms	faster_model	latency_ratio	amount_btc
Adapting after drift in	DRIFT_ADAPTATION	0.374	6.83	25.1	HST	66.9	0.00260
Adapting after drift in	DRIFT_ADAPTATION	0.367	9.06	22.2	HST	60.5	0.00225
Adapting after drift in	DRIFT_ADAPTATION	0.211	8.79	21.1	HST	100	214
Adapting after drift in	DRIFT_ADAPTATION	0.228	7.41	17.1	HST	75.3	0.00505
Adapting after drift in	DRIFT_ADAPTATION	0.179	6.57	15.0	HST	83.8	0.488
Adapting after drift in	DRIFT_ADAPTATION	0.296	6.78	21.3	HST	72.0	0.0161
Adapting after drift in	DRIFT_ADAPTATION	0.144	8.78	21.1	HST	147	0.797
Adapting after drift in	DRIFT_ADAPTATION	0.299	8.52	20.7	HST	69.1	0.00822
Adapting after drift in	DRIFT_ADAPTATION	0.295	11.2	22.8	HST	77.4	0.0309
Adapting after drift in	DRIFT_ADAPTATION	0.201	9.43	19.1	HST	95.2	1.25
Adapting after drift in	DRIFT_ADAPTATION	0.333	7.54	24.0	HST	71.9	0.00424

